

AURIX

Microcontroller Infineon Software Architecture

User's Manual

FLS Driver

Release v4.4

2017-04

Edition 2017-04

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2017 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Document Change History		
Date	Version	Change Description
2017-04-28	4.4	Added Section 4.4.14.6. Updated sections 4.5.3, 6.2.16, 6.2.17, 8, and Figure 2. Removed the suspend operation usage limitation from section 10.2. Above changes is related to new Dem report for Suspend timeout.
2016-05-04	4.3	Updated for JIRA Id:0000051261-231 (I/O mode updated for USER Mode support for Section 6.2) Added API Fls_17_Pmu_VerifySectorErase
2015-11-10	4.2	Updated Section 6.2.11 and 6.2.12 for JIRA 0000051105-17
2015-09-24	4.1	Updated Sections 6.2.2, 6.2.3, 6.2.6, 6.2.10 for JIRA 0000051100-178
2015-07-21	4.0	Updated as per Review Comments REV_008095.
2015-07-08	3.9	Updated Section 10.2 for JIRA 0000051129-11 and Section 1.1, 4.3.13, 4.4.17.2 to provide information for device specific details.
2015-04-23	3.8	Updated as per Review Comments REV_007541
2015-04-21	3.7	Updated Sections 2.1.3 and 9.1 for JIRA 0000051129-4 Updated Section 2.2 for JIRA 0000051023-33
2015-03-02	3.6	Updated Fls_17_Pmu_SetMode for JIRA 0000051050-2. Updated for TC21x derivatives.
2014-12-02	3.5	Updated for UTP AI00251090 - I/O Mode Information
2014-10-20	3.4	Updated as per review comments of REV_006302.
2014-10-14	3.3	Updated as per UTP AI00249919 – Updated Fls_17_Pmu_Init details
2014-09-16	3.2	Updated as per review comments of REV_006171.
2014-09-05	3.1	Updated for TC22x (Read and ECC correction wait states). Updated default value of FlsEraseSuspendTimeout.
2014-08-26	3.0	Updated as per review comments of REV_006068
2014-08-23	2.9	Updated as per UTP AI00247498 – Updated Fls_17_Pmu_Cancel details
2014-07-16	2.8	Added section 6.3 for debug macro information.
2014-05-30	2.7	Updated for UTP AI00239662. Added API Fls_17_Pmu_ControlTimeoutDet.
2014-05-21	2.6	Updated as per review comments REV_005309
2014-02-24	2.5	Updated for Aurix EP devices
2013-11-11	2.4	Updated for adding modified SFR information for all the configuration parameters
2013-09-30	2.3	Updated as per review comments of REV_004844.
2013-09-24	2.2	Updated for UTP AI00219555
2013-09-11	2.1	Updated as per review comments of REV_004751
2013-08-01	2.0	Updated the description of Fls_17_Pmu_Erase API, TC29x changes, removed Margin parameter from Fls_17_CompareWordsSync API.
2013-06-27	1.9	Updated for TC26x support
2013-04-30	1.8	Updated review comments REV_004474
2013-04-26	1.7	Updated Ch 6.2.1.15
2013-04-05	1.6	Updated Suspend/Resume Erase feature and File structure
2012-12-13	1.5	Updated for AI00133839
2012-11-22	1.4	Updated for AI00114543
2012-10-30	1.3	Updated for AI00108519

CONFIDENTIAL

2012-08-10	1.2	REV_003656: Updated review comments , Updated Exclusive area
2012-06-21	1.1	Updated for TC27x and IFX specific APIs
2011-11-09	1.0	Updated PublishedParameters for vendor ID, Baselined
2011-10-25	0.4	REV_003161: Updated Ch 4.2, Ch 5, Removed FLS_CR_xxx Ids
2011-10-05	0.3	Updated API names for vendor ID, Changes for TC2Dx
2011-04-13	0.2	Updated limitations section
2011-03-22	0.1	Initial Version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents	Page
1 Introduction	12
1.1 Scope	12
1.2 Abbreviations.....	14
1.3 References	15
2 Driver Overview	15
2.1 Software Hardware Mapping.....	15
2.1.1 Data Flash Unit.....	16
2.1.2 Linking Different User Configurations during Run Time	16
2.1.3 Hints about copy of APIs Fls_WriteCmdCycles and Fls_EraseCmdCycles	17
2.2 Software Driver Description	17
3 File structure.....	18
4 Configuration Documentation	20
4.1 Configuration Concept	20
4.1.1 Configuration Class	20
4.1.2 Configuration Variant	21
4.1.3 How to read the Configuration Class field.....	21
4.2 Configuration Parameter Variance.....	21
4.3 Container: FlsGeneral	21
4.3.1 FlsAcLoadOnJobStart	21
4.3.2 FlsCancelApi	22
4.3.3 FlsCompareApi.....	22
4.3.4 FlsDevErrorDetect.....	23
4.3.5 FlsGetJobResultApi	23
4.3.6 FlsResetSfrAtInit	24
4.3.7 FlsGetStatusApi	24
4.3.8 FlsSetModeApi	25
4.3.9 FlsUseInterrupts	25
4.3.10 FlsVersionInfoApi	26
4.3.11 FlsDriverIndex	26
4.3.12 FlsBaseAddress	26
4.3.13 FlsTotalSize.....	27
4.3.14 FlsPBFixedAddress.....	29
4.3.15 FlsDebugSupport	30
4.4 Container: FlsConfigSet	30
4.4.1 FlsWaitStateRead	30
4.4.2 FlsWaitStateErrorCorrection	31
4.4.3 FlsCallCycle	31
4.4.4 FlsDefaultMode	32
4.4.5 FlsAcErase	32
4.4.6 FlsAcWrite	33
4.4.7 FlsJobEndNotification	33
4.4.8 FlsJobErrorNotification	34
4.4.9 FlsMaxReadFastMode	34
4.4.10 FlsMaxReadNormalMode	35
4.4.11 FlsMaxWriteFastMode	35
4.4.12 FlsMaxWriteNormalMode.....	36
4.4.13 FlsProtection	36
4.4.14 Container: FlsDemEventParameterRefs.....	36
4.4.14.1 FLS_E_COMPARE_FAILED	37
4.4.14.2 FLS_E_ERASE_FAILED	37
4.4.14.3 FLS_E_READ_FAILED.....	37
4.4.14.4 FLS_E_WRITE_FAILED	38
4.4.14.5 FLS_E_UNEXPECTED_FLASH_ID	38
4.4.14.6 FLS_E_SUSPND_RESUME_TIMEOUT	39
4.4.15 Container: FlsExternalDriver	39
4.4.15.1 FlsSpiReference	39

Table of Contents	Page
4.4.16 Container: FlsSectorList.....	39
4.4.17 Container: FlsSector	39
4.4.17.1 FlsSectorStartAddress	40
4.4.17.2 FlsSectorSize	40
4.4.17.3 FlsPageSize	42
4.4.17.4 FlsNumberOfSectors.....	43
4.5 Container: FlsIfxSpecificConfig	43
4.5.1 FlsStateVarStruct	43
4.5.2 FlsUseEraseSuspend	44
4.5.3 FlsEraseSuspendTimeout.....	44
4.5.4 FlsIllegalStateNotification.....	45
4.6 Container: FlsPublishedInformation	45
4.6.1 FlsAcSizeErase	45
4.6.2 FlsAcSizeWrite	46
4.6.3 FlsEraseTime	46
4.6.4 FlsWriteTime	46
4.6.5 FlsErasedValue	46
4.6.6 FlsSpecifiedEraseCycles	46
4.6.7 FlsAcLocationErase	47
4.6.8 FlsAcLocationWrite	47
4.6.9 FlsExpectedHwld	47
4.7 Container: CommonPublishedInformation	47
4.7.1 ArMajorVersion.....	47
4.7.2 ArMinorVersion.....	48
4.7.3 ArPatchVersion	48
4.7.4 SwMajorVersion	48
4.7.5 SwMinorVersion	48
4.7.6 SwPatchVersion	48
4.7.7 ModuleId.....	49
4.7.8 VendorId.....	49
4.7.9 VendorApiInfix.....	49
4.7.10 Release	49
4.8 Derived Configuration Parameters.....	50
4.8.1 FLS_IFX_FEE_USED	50
4.8.2 FLS_SECTOR_SIZE.....	50
4.8.3 FLS_17_PMU_CONFIG_COUNT	51
5 Published parameters.....	51
5.1 VendorId.....	51
5.2 ModuleId.....	51
5.3 SwMajorVersion	51
5.4 SwMinorVersion	52
5.5 SwPatchVersion	52
5.6 AR_RELEASE_MAJOR_VERSION.....	52
5.7 AR_RELEASE_MINOR_VERSION	52
5.8 AR_RELEASE_REVISION_VERSION	53
6 API Documentation	53
6.1 API Type Definitions.....	53
6.1.1 Type definition Fls_ConfigType.....	53
6.1.2 Type definition MemIf_StatusType.....	53
6.1.3 Type definition MemIf_JobResultType	54
6.1.4 Type definition MemIf_DataType	54
6.1.5 Type definition Fls_LengthType	54
6.1.6 Type definition Fls_AddressType	55
6.2 API Functions.....	55
6.2.1 Fls_17_Pmu_Init	55
6.2.2 Fls_17_Pmu_Erase.....	56
6.2.3 Fls_17_Pmu_Write.....	57

Table of Contents	Page
6.2.4 Fls_17_Pmu_Cancel	59
6.2.5 Fls_17_Pmu_MainFunction	59
6.2.6 Fls_17_Pmu_Read	61
6.2.7 Fls_17_Pmu_Compare	62
6.2.8 Fls_17_Pmu_SetMode.....	63
6.2.9 Fls_17_Pmu_GetVersionInfo	63
6.2.10 Fls_17_Pmu_ReadWordsSync	64
6.2.11 Fls_17_Pmu_GetStatus	65
6.2.12 Fls_17_Pmu_GetJobResult	65
6.2.13 Fls_17_Pmu_CompareWordsSync.....	66
6.2.14 Fls_17_Pmu_VerifyErase	67
6.2.15 Fls_17_Pmu_VerifySectorErase	67
6.2.16 Fls_17_Pmu_SuspendErase	68
6.2.17 Fls_17_Pmu_ResumeErase	69
6.2.18 Fls_17_Pmu_GetBankStatus.....	70
6.2.19 Fls_17_Pmu_CancelNonEraseJobs	70
6.2.20 Fls_17_Pmu_GetNotifCaller	71
6.2.21 Fls_17_Pmu_GetOperStatus	72
6.2.22 Fls_17_Pmu_ControlTimeoutDet.....	72
6.3 Debug Macros.....	73
6.4 Interrupt Handling.....	73
6.4.1 Fls_17_Pmu_Isr	73
6.5 Exclusive Area (EA) Information	74
6.5.1 Init.....	74
6.5.2 Write.....	74
6.5.3 Erase	75
6.5.4 Main.....	75
6.5.5 Resume Erase.....	75
7 Data Description.....	75
8 Error Classification	76
9 Example Usage.....	77
9.1 Configuration of the Driver	77
9.2 Using the Flash Services	77
9.2.1 Initializing the Driver	77
9.2.2 Flash Operations	77
10 Assumptions and Limitations.....	78
10.1 Assumptions and Deviations from the Software Specification.....	78
10.2 Limitations	78
10.3 Hardware Features Not Supported	78

List of Figures

Page

Figure 1	SW HW Mapping.....	16
Figure 2	Driver Description.....	18
Figure 3	Driver File Structure	19
Figure 4	Interrupt Handling.....	73

List of Tables
Page

Table 1	Variance of peripheral units/features	12
Table 2	Abbreviations used in this document	14
Table 3	File Contents	19
Table 4	Configuration Class Example 1	21
Table 5	Configuration Class Example 2	21
Table 6	Flash Access Code Load Address configuration	21
Table 7	FlsCancelApi Configuration	22
Table 8	FlsCompareApi Configuration	22
Table 9	Development Error Tracer configuration	23
Table 10	FlsGetJobResultApi Configuration	23
Table 11	FlsResetSfrAtInit Configuration	24
Table 12	FlsGetStatusApi Configuration	24
Table 13	FlsSetModeApi Configuration	25
Table 14	Interrupts Enable configuration	25
Table 15	FlsVersionInfoApi Configuration	26
Table 16	FlsDriverIndex Configuration	26
Table 17	Flash memory Base Address configuration	26
Table 18	Flash Memory Total Size configuration	27
Table 19	Range of Fls Total Size for various devices	27
Table 20	Fixed Address usage for FLS configuration	29
Table 21	Debug support for FLS configuration	30
Table 22	FlsWaitStateRead Configuration	30
Table 23	FlsWaitStateErrorCorrection Configuration	31
Table 24	FlsCallCycle Configuration	31
Table 25	FlsDefaultMode Configuration	32
Table 26	FlsAcErase Configuration	32
Table 27	FlsAcWrite Configuration	33
Table 28	FlsJobEndNotification API Configuration	33
Table 29	FlsJobErrorNotification API Configuration	34
Table 30	FlsMaxReadFastMode Configuration	34
Table 31	FlsMaxReadNormalMode Configuration	35
Table 32	FlsMaxWriteFastMode Configuration	35
Table 33	FlsMaxWriteNormalMode Configuration	36
Table 34	FlsProtection Configuration	36
Table 35	FLS_E_COMPARE_FAILED	37
Table 36	FLS_E_ERASE_FAILED	37
Table 37	FLS_E_READ_FAILED	37
Table 38	FLS_E_WRITE_FAILED	38
Table 39	FLS_E_UNEXPECTED_FLASH_ID	38
Table 40	FlsSpiReference Configuration	39
Table 41	Start Address of the Fls Sector	40
Table 42	Size of the Fls Sector	40
Table 43	Range of Fls Sector Size for various devices	40
Table 44	Size of the Fls Page	42
Table 45	Number of sectors	43
Table 46	FlsStateVarStruct Configuration	43
Table 47	FlsUseEraseSuspend Configuration	44
Table 48	FlsEraseSuspendTimeout Configuration	44
Table 49	FlsIllegalStateNotification Configuration	45
Table 50	Erase Access Code size	45
Table 51	Write Access Code size	46
Table 52	Erase Time	46
Table 53	Write time	46
Table 54	Erased Value	46
Table 55	Erase Cycles Supported	46
Table 56	FLS Erase Command cycles location	47
Table 57	FLS Write Command cycles location	47
Table 58	Hardware ID	47

List of Tables

Page

Table 59	AUTOSAR Major Version.....	47
Table 60	AUTOSAR Minor Version.....	48
Table 61	AUTOSAR Patch Version	48
Table 62	Software Major Version	48
Table 63	Software Minor Version	48
Table 64	Software Patch Version.....	48
Table 65	Module Id.....	49
Table 66	Vendor Id.....	49
Table 67	Vendor API Infix	49
Table 68	Release	49
Table 69	FLS_IFX_FEE_USED Configuration	50
Table 70	FLS_SECTOR_SIZE Configuration	50
Table 71	FLS_17_PMU_CONFIG_COUNT Configuration	51
Table 72	VendorId.....	51
Table 73	ModuleId.....	51
Table 74	SwMajorVersion	51
Table 75	SwMinorVersion	52
Table 76	SwPatchVersion	52
Table 77	FLS Driver AUTOSAR Requirement Major Version.....	52
Table 78	FLS Driver AUTOSAR Requirement Minor Version.....	52
Table 79	FLS Driver AUTOSAR Requirement Patch Version	53
Table 80	Type definition Fls_ConfigType.....	53
Table 81	Type definition MemIf_Status Type.....	53
Table 82	Type definition MemIf_JobResultType.....	54
Table 83	Type definition MemIf_DataType	54
Table 84	Type definition Fls_LengthType	54
Table 85	Type definition Fls_AddressType.....	55
Table 86	Specification of API Fls_17_Pmu_Init.....	55
Table 87	Specification of API Fls_17_Pmu_Erase	56
Table 88	Specification of API Fls_17_Pmu_Write	57
Table 89	Specification of API Fls_17_Pmu_Cancel	59
Table 90	Specification of API Fls_17_Pmu_MainFunction	59
Table 91	Specification of API Fls_17_Pmu_Read	61
Table 92	Specification of API Fls_17_Pmu_Compare.....	62
Table 93	Specification of API Fls_17_Pmu_SetMode	63
Table 94	Specification of API Fls_17_Pmu_GetVersionInfo.....	63
Table 95	Specification of API Fls_17_Pmu_ReadWordsSync.....	64
Table 96	Specification of API Fls_17_Pmu_GetStatus.....	65
Table 97	Specification of API Fls_17_Pmu_GetJobResult.....	65
Table 98	Specification of API Fls_17_Pmu_CompareWordsSync	66
Table 99	Specification of API Fls_17_Pmu_VerifyErase	67
Table 100	Specification of API Fls_17_Pmu_VerifySectorErase	67
Table 101	Specification of API Fls_17_Pmu_SuspendErase	68
Table 102	Specification of API Fls_17_Pmu_ResumeErase.....	69
Table 103	Specification of API Fls_17_Pmu_GetBankStatus	70
Table 104	Specification of API Fls_17_Pmu_CancelNonEraseJobs.....	70
Table 105	Specification of API Fls_17_Pmu_GetNotifCaller	71
Table 106	Specification of API Fls_17_Pmu_GetOperStatus	72
Table 107	Specification of API Fls_17_Pmu_ControlTimeoutDet	72
Table 108	Specification of API Fls_17_Pmu_Isr.....	73
Table 109	Specification of EA Init	74
Table 110	Specification of EA Write.....	74
Table 111	Specification of EA Erase.....	75
Table 112	Specification of EA Main	75
Table 113	Specification of EA Resume Erase	75
Table 114	Global Data Description	75
Table 115	Error Classification	76

List of Code Listings

Page

Code Listing 1	Sample Options Listing	17
Code Listing 2	Driver Initialization	77
Code Listing 3	Flash Operations	77

1 Introduction

This User Manual provides information regarding the functionality and API implementation for the Flash driver for AUTOSAR Release 4.0 Rev3 [2]. The FLASH Driver from AUTOSAR is mainly intended to be used for EEPROM Emulation. So the FLASH driver implementation is only for the Data Flash unit of the AURIX FLASH Module. The program flash is not considered. The driver provides services for reading from and writing to, erasing Data flash unit.

This User Manual is intended to help the user to get acquainted with the Flash Driver implementation for the AURIX Hardware Platform. It is the documentation that describes how to use the Flash Driver.

1.1 Scope

This document addresses the following features of the Flash Driver implementation,

AURIX microcontroller HW modules used to realize the Driver

File structure of the Driver

Configuration parameters of the Driver

Parameters published by the Driver

API Specification

Interrupt Handling

Common use cases with code snippets.

Limitations and Assumptions

This documentation applies to the FLS Driver Variant: Variant PB

This document addresses the FLS driver implementation and the following table gives the available units/features in the device.

Note: This FLS driver is implemented assuming that the FEE driver which uses this driver uses the double sector algorithm for EEPROM emulation.

For eg. In TC27x there are 48 logical sectors of 8KB each (total size is 384KB) as shown in the Table 1, which are divided into 2 large sectors of size 192KB each. From the FLS driver's perspective these two sectors are the actual Dflash sectors. This logical sector formation is to aid in the Double sector algorithm implemented by the FEE driver. Hence one sector of size 192 KB can be erased at a time.

Note: In order to support marking options with reduced DF_EEPROM size, the configuration parameter FlsTotalSize has to be configured accordingly. Since the logical sector size is 8KB and minimum 2 logical sectors are required for the implementation of double sector algorithm, the value of FlsTotalSize should be multiple of 16KB.

Table 1 Variance of peripheral units/features

Microcontroller \ Features	No. of sectors	DFLASH Sector Size
TC299TP_128F300S	96	8 KB
TC299T_128F300S		
TC298TP_128F300L		
TC297TP_128F300S		
TC297T_128F300S		
TC290TP_128F300		
TC299TX_128F300S		
TC299TY_128F300S		
TC297TX_128F300S		
TC297TY_128F300S		
TC297T_96F300S		

Microcontroller \ Features	No. of sectors	DFLASH Sector Size
TC297TP_96F300S TC297TA_128F300S TC297TB_128F300S TC297TA_64F300S X		
TC290TC_96F300 TC297TC_96F300S TC299TC_96F300S	24	8 KB
TC277TP_64F200S TC277T_64F200S TC275T_64F200W TC275TP_64F200W	48	8 KB
TC277TC_64F200S TC275TC_64F200W	18	8 KB
TC267D_40F200S TC265D_40F200W TC264D_40F200W TC260D_40F200 TC264DA_40F200W	12	8 KB
TC265DC_40F200W TC264DC_40F200W	8	8 KB
TC237LP_32F200S TC234LP_32F200F TC233LP_32F200F TC237L_32F200S TC234L_32F200F TC233L_32F200F TC234LA_32F200F TC234LX_32F200F TC237LA_32F200S TC237LX_32F200S TC234LP_16F200F TC233L_16F200F TC233LP_16F200F TC233LC_24F133F	16	8 KB
TC224L_16F133F TC223L_16F133F TC222L_16F133F TC224S_16F133F TC223S_16F133F TC222S_16F133F TC223L_12F133F TC222L_12F133F	12	8 KB
TC222S_12F133F TC223S_12F133F	8	8 KB

Microcontroller \ Features	No. of sectors	DFLASH Sector Size
TC214L_8F133F TC213L_8F133F TC212L_8F133F TC214S_8F133F TC213S_8F133F TC214L_8F133F TC213L_8F133F TC212L_8F133F	8	8 KB
TC212S_8F133F	6	8 KB

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 2](#).

Table 2 Abbreviations used in this document

Abbreviation	Explanation
API	Application Programming Interface
AURIX	AUtomotive Realtime Integrated NeXt Generation Architecture
AUTOSAR	AUTomotive Open System Architecture
BSW	Basic Software
CPU	Central Processing Unit
DET	Development Error Tracer
DFLASH	Data Flash
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read Only Memory
EP	Efficiency Platform
FEE	Flash EEPROM Emulation
FLS	Flash
FSI	Flash Standard Interface
HW	Hardware
IFX	Infineon Technologies
KB	KiloByte
MC-ISAR	Microcontroller Infineon Software Architecture
MCAL	MicroController Abstraction Layer
MCU	MicroController Unit
μs	Microsecond
μC	Microcontroller
RAM	Random Access Memory
ROM	Read Only Memory
SFR	Special Function Register
SRN	Service Request Node
SW	Software
TBD	To Be Defined

tDFECC	Data Flash ECC Delay
tDF	Data Flash access delay
WL	Word-line

1.3 References

This section lists down all relevant documents for this User Manual.

Refer to the latest version for the documents without version information.

- [1] Target Data Sheet / user manual:
 tc27xC_um_v2.0OM.pdf
 tc29xB_um_v1.2_2.pdf
 tc26xB_um_v1.2.pdf
 tc23x_tc22x_um_v1.0.pdf
- [2] Software Specification
 AUTOSAR_SWS_FLSDriver.pdf, V4.0 Rev3
- [3] MC-ISAR_AURIX_SAS.doc v15.6, Aurix_MC-ISAR_MEM_SAS.doc v1.2

2 Driver Overview

The microcontroller of the AURIX family provides data flash memory which can be used for emulation of EEPROM data. The Flash driver from AUTOSAR is intended for use by the EEPROM Emulation software. This implementation handles only the Data Flash unit.

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the Hardware peripherals and their interaction with the Software Drivers used in AURIX.

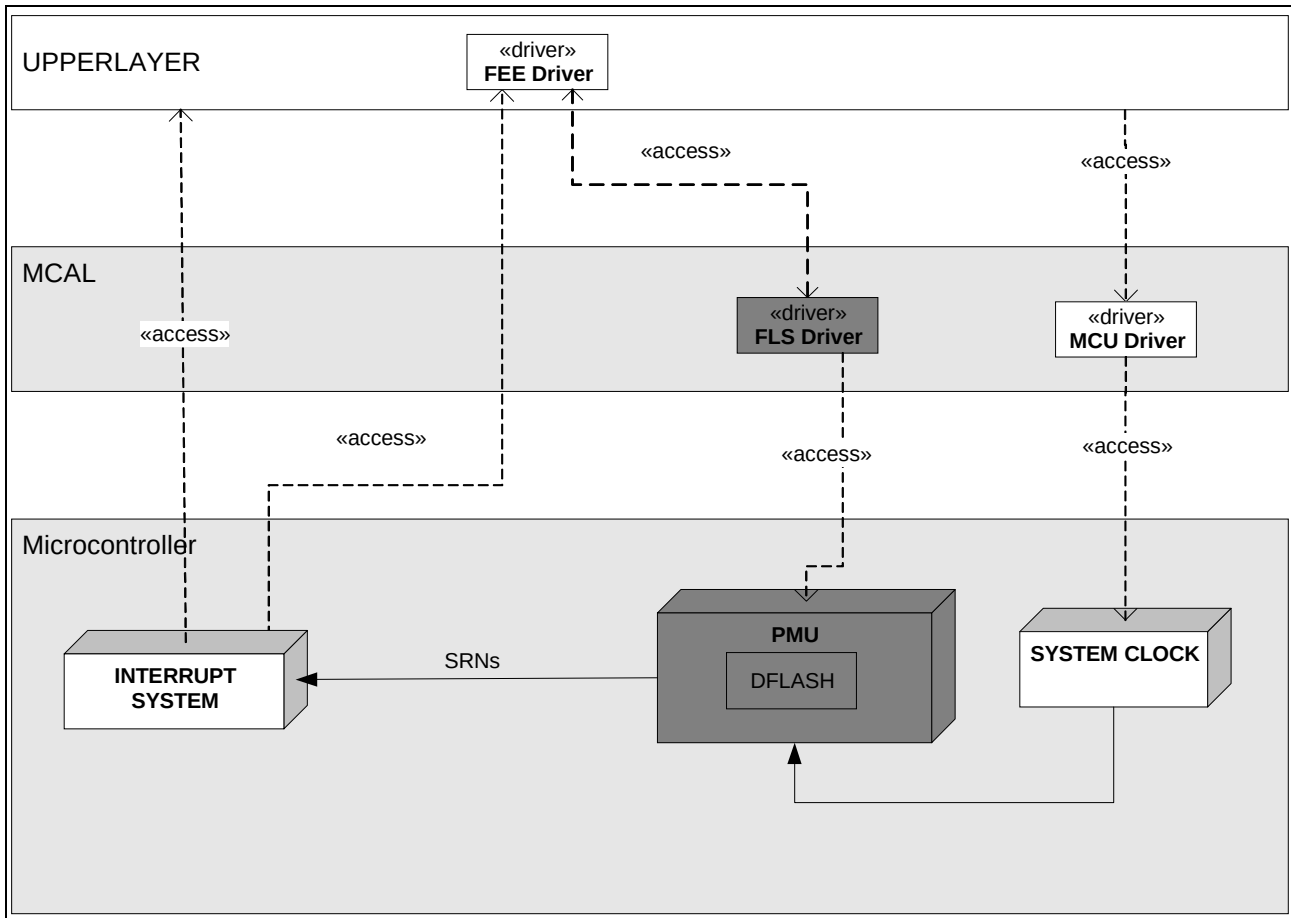


Figure 1 SW HW Mapping

2.1.1 Data Flash Unit

The Data Flash unit of the AURIX has the following features,

- Usable for data storage with EEPROM functionality
- 8/32 byte of program interface
 - 8/32 bytes are programmed into one DFLASH page by one step/command depending on the size and page start address in the flash
- 64-bit read interface (no burst transfers)
 - Dynamic correction of single-bit, double-bit and triple-bit errors during read access
 - Detection of multiple-bit errors
- Fixed sector architecture
 - 48 sectors of equal size in one Bank.
- Status and error reporting by status flags and interrupt

2.1.2 Linking Different User Configurations during Run Time

This FLS module provides user to get away with linking problems by allocating RAM for all the global variables in a structure. The address of the global variable structure (RAM) is configurable via the global configuration structure of FLS.

2.1.3 Hints about copy of APIs Fls_WriteCmdCycles and Fls_EraseCmdCycles

If FlsAcLoadOnJobStart is true, then the functions Fls_WriteCmdCycles and Fls_EraseCmdCycles will be copied to RAM by the FLS driver, before they are executed. For this purpose, the user has to reserve 300 bytes for loading Write command sequence and 200 bytes for loading Erase command sequence to RAM. For example, refer [Code Listing 1](#) for TASKING compiler.

Memory sections are provided for the command cycle functions. Using start and end addresses of these sections, the size of the function is calculated and copied to RAM.

Code Listing 1 Sample Options Listing

```

01:  section_layout mpe:vtc:linear
02:  {
03:      group FLS_AC_ERASE_SOURCE (ordered, contiguous, run_addr=0x80008100)
04:      {
05:          select ".text.FLS_AC_ERASE_SOURCE";
06:      }
07:  }
08:  section_layout mpe:vtc:linear
09:  {
10:      group FLS_AC_WRITE_SOURCE (ordered, contiguous, run_addr=0x80008300)
11:      {
12:          select ".text.FLS_AC_WRITE_SOURCE";
13:      }
14:  }
15:
16:  section_layout spe:tc:linear
17:  {
18:      group FLS_AC_WRITE (ordered, run_addr = 0x70101100)
19:      {
20:          reserved "FLS_AC_WRITE" (alloc_allowed = absolute, size = 300);
21:      }
22:  }
23:  section_layout spe:tc:linear
24:  {
25:      group FLS_AC_ERASE (ordered, run_addr = 0x7010122C)
26:      {
27:          reserved "FLS_AC_ERASE" (alloc_allowed = absolute, size = 200);
28:      }
29:  }

```

2.2 Software Driver Description

The [Figure 2](#) shows the Application Programming Interface (API) for the Flash Driver and its interaction with other software Drivers.

– Development Error Tracer (DET) is required if DET is enabled for the FlashDriver.

The APIs of the Driver are explained in detail in [chapter 6.2](#)

The Configuration Data of the Driver is also depicted in the figure. The individual containers and their configuration parameters are explained in detail in [chapter 6.1](#).

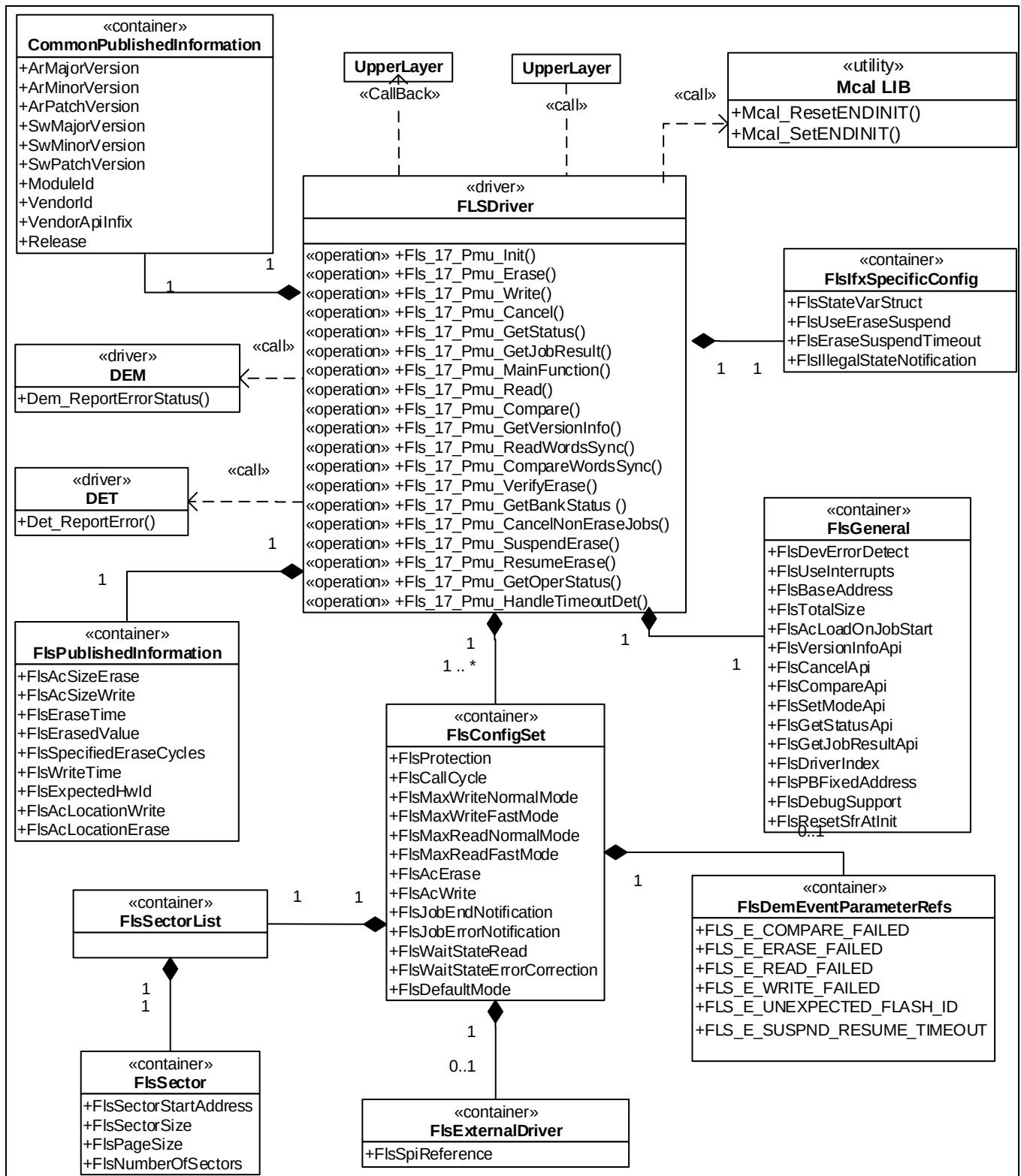


Figure 2 Driver Description

3 File structure

This section provides details about the FLASH driver files and other related files.

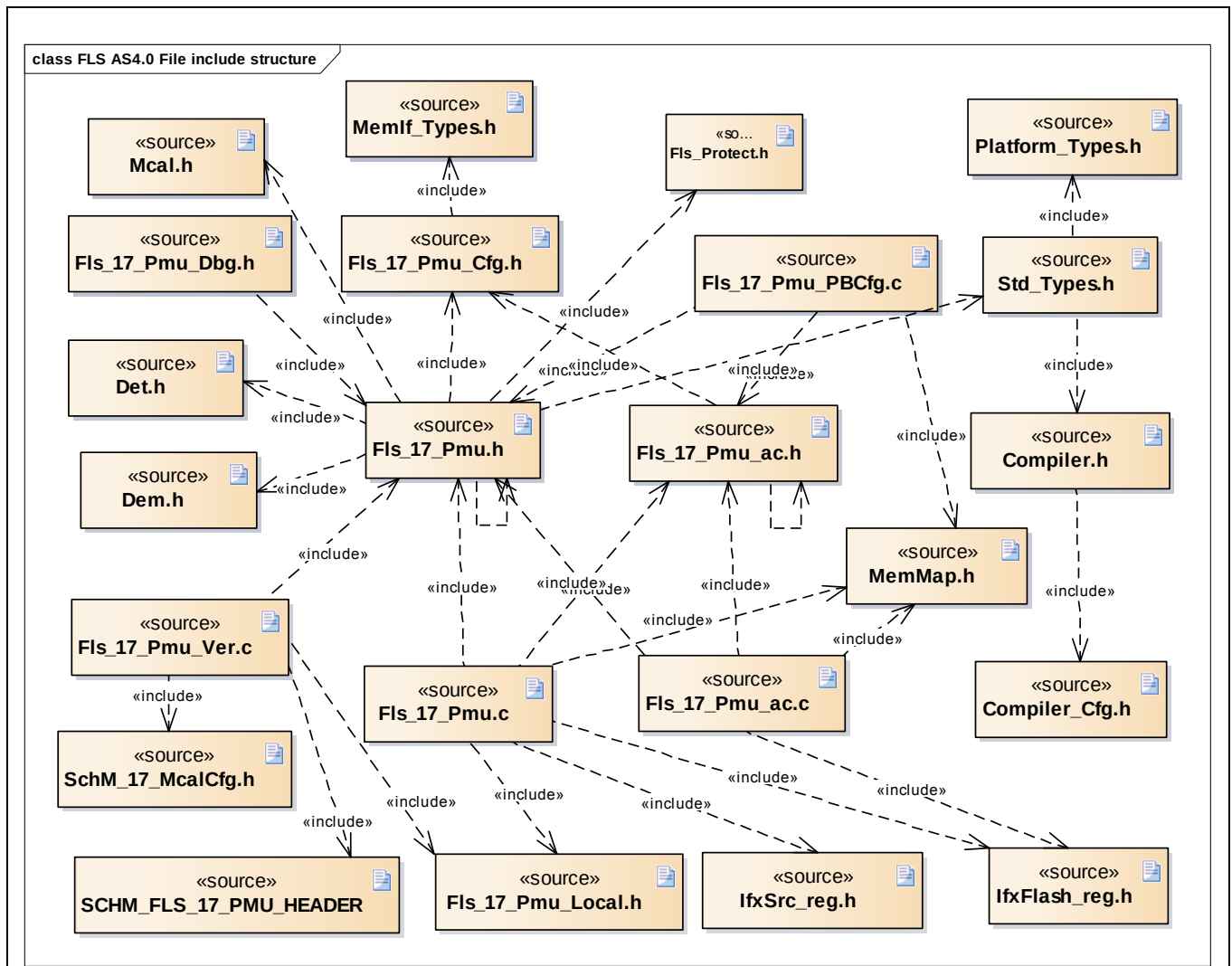


Figure 3 Driver File Structure

Table 3 File Contents

File Name	File Contents
Platform_Types.h	Platform Dependencies are declared here.
Compiler.h	Compiler Dependencies are declared here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code.
Std_Types.h	Standard Data types to be used are declared here.
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections
Fls_17_Pmu.h	This header file exports macros, type definitions and function prototypes for the Flash Driver
Fls_17_Pmu_Local.h	FLS module internal header file
Fls_17_Pmu.c	This file contains functionality of the FLS driver
Fls_17_Pmu_Ver.c	Autosar version specific functionality of the FLS driver
Fls_17_Pmu_ac.c	Command cycles for Flash operations
Fls_17_Pmu_ac.h	Header file for macros used by Fls_17_Pmu_ac.c

Fls_17_Pmu_Cfg.h	Contains Driver pre-compile configuration parameters
Fls_17_Pmu_Dbg.h	Provides debug variables of FLS driver
Fls_17_Pmu_PBCfg.c	Contains Driver post-build configuration parameters
Fls_17_Cbk.h	Provides FLS driver's callback functionalities
IfxFlash_reg.h	Includes the register definition for Flash IP's SFRs
IfxSrc_reg.h	Includes the register definition for interrupt SFRs
Mcal.h	Provides library functions for MCAL layer
Det.h	Provides the functionality of Development Error Tracer
Dem.h	Provides the functionality of Production Error Reporting
SchM_17_MCalCfg.h	This file defines the IFX MCAL Module specific SchM header file names
SCHM_FLS_17_PMU_HEADER	This file (SchM_Fls_17_Pmu.h) provides the functionality for critical sections used by FLS driver
MemIf_Types.h	Provides the memory interface layer's type definitions

4 Configuration Documentation

This chapter summarizes

Configuration concept used for structuring the Driver

- Configuration parameter variance w.r.t supported HW

All configuration parameters and their description

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

Compiling,

Linking and

Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be frozen before compilation and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

Note: The Flash Driver is delivered as a Variant PB.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Note: Topics [4.1.1](#), [4.1.2](#) should be read for understanding the configuration class field

Table 4 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, and Variant PC.

Table 5 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

4.2 Configuration Parameter Variance

This section describes the variance in the configuration parameter w.r.t supported microcontroller.

Refer Chap. [4.4.17](#) and [4.3.13](#).

4.3 Container: FlsGeneral

4.3.1 FlsAcLoadOnJobStart

Table 6 Flash Access Code Load Address configuration

Syntax:	FlsAcLoadOnJobStart
Source:	This parameter is defined by AUTOSAR, refer [2] .
File:	Fls_17_Pmu_Cfg.h

Range:	Values : STD_ON - Access Code load to RAM on Job start is enabled STD_OFF - Access Code is not loaded to RAM but directly executed from Flash Default Value : STD_OFF		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If this Macro is Enabled then, Erase access code will be loaded in RAM during Fls_Erase API call and unloaded after completion or cancellation of job. Similarly Write access code will be loaded in RAM during Fls_Write API call and unloaded after completion or cancellation of job. If this Macro is Disabled then, Write and Erase flash access code will be executed from Flash.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.2 FlsCancelApi

Table 7 FlsCancelApi Configuration

Syntax:	FlsCancelApi		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Fls_17_Pmu_Cancel API is enabled STD_OFF - Fls_17_Pmu_Cancel API is disabled Default Value : STD_OFF		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This is used to enable/disable the Fls_17_Pmu_Cancel API		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.3 FlsCompareApi

Table 8 FlsCompareApi Configuration

Syntax:	FlsCompareApi		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		

Range:	Values : STD_ON - Fls_17_Pmu_Compare API is enabled STD_OFF - Fls_17_Pmu_Compare API is disabled Default Value : STD_OFF		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This is used to enable/disable the Fls_17_Pmu_Compare API		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.4 FlsDevErrorDetect

Table 9 Development Error Tracer configuration

Syntax:	FlsDevErrorDetect		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Development error detection enabled STD_OFF - Development error detection disabled Default Value : STD_ON		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the development error detection and Reporting.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.5 FlsGetJobResultApi

Table 10 FlsGetJobResultApi Configuration

Syntax:	FlsGetJobResultApi		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Fls_17_Pmu_GetJobResult API is enabled STD_OFF - Fls_17_Pmu_GetJobResult API is disabled Default Value : STD_OFF		

Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This is used to enable/disable the Fls_17_Pmu_GetJobResult API		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.6 FlsResetSfrAtInit

Table 11 FlsResetSfrAtInit Configuration

Syntax:	FlsResetSfrAtInit		
Source:	This parameter is microcontroller specific		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Feature to Reset SFR to default values in Fls_17_Pmu_Init function enabled STD_OFF - Feature to Reset SFR default values in Fls_17_Pmu_Init function disabled Default Value : STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This Parameter indicates whether to enable or disable the Reset of SFR to default values in Fls_17_Pmu_Init function.		
SFRs Modified	FLASH0_FCON, SRC_PMU00, FLASH0_MARD		
Scope:	Driver		
Dependency:	None		

4.3.7 FlsGetStatusApi

Table 12 FlsGetStatusApi Configuration

Syntax:	FlsGetStatusApi		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Fls_17_Pmu_GetStatus and Fls_17_Pmu_GetBankStatus APIs are enabled STD_OFF - Fls_17_Pmu_GetStatus and Fls_17_Pmu_GetBankStatus APIs are disabled Default Value : STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--

	Post build	--	--
Description:	This is used to enable/disable Fls_17_Pmu_GetStatus and Fls_17_Pmu_GetBankStatus APIs.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.8 FlsSetModeApi

Table 13 FlsSetModeApi Configuration

Syntax:	FlsSetModeApi		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Fls_17_Pmu_SetMode API is enabled STD_OFF - Fls_17_Pmu_SetMode API is disabled Default Value : STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This is used to enable/disable the Fls_17_Pmu_SetMode API		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.9 FlsUseInterrupts

Table 14 Interrupts Enable configuration

Syntax:	FlsUseInterrupts		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Interrupts enabled STD_OFF - Interrupts disabled Default Value : STD_OFF		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the interrupts.		
SFRs Modified	FLASH0_FCON, SRC_PMU00		
Scope:	Driver		
Dependency:	None		

4.3.10 FlsVersionInfoApi

Table 15 FlsVersionInfoApi Configuration

Syntax:	FlsVersionInfoApi		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : STD_ON - Fls_17_Pmu_GetVersionInfo API is enabled STD_OFF - Fls_17_Pmu_GetVersionInfo API is disabled Default Value : STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This is used to enable/disable the Fls_17_Pmu_GetVersionInfo API		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.11 FlsDriverIndex

Table 16 FlsDriverIndex Configuration

Syntax:	FlsDriverIndex		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Values : 0 to 254 Default Value : 0		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is used to assign an index to the FlsDriver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.12 FlsBaseAddress

Table 17 Flash memory Base Address configuration

Syntax:	FlsBaseAddress		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value: Data flash base address is 0xAF000000.		

	This parameter is FIXED and not editable		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter defines the lower boundary for read/write/erase jobs.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.13 FlsTotalSize

Table 18 Flash Memory Total Size configuration

Syntax:	FlsTotalSize		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Refer Table 19		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is used to calculate the the upper boundary for read/write/erase jobs. This parameter should be updated accordingly for marking option with reduced DF_EEPROM size. The value of this parameter should be multiple of 16384 (16KB).		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

Table 19 Range of Fls Total Size for various devices

Devices	Default	Value
TC299TP_128F300S	786432	16384 to 786432
TC299T_128F300S		
TC298TP_128F300L		
TC297TP_128F300S		
TC297T_128F300S		
TC290TP_128F300		
TC299TX_128F300S		
TC299TY_128F300S		
TC297TX_128F300S		
TC297TY_128F300S		
TC297T_96F300S		

TC297TP_96F300S		
TC290TC_96F300 TC297TC_96F300S TC299TC_96F300S	196608	16384 to 196608
TC277TP_64F200S TC277T_64F200S TC275T_64F200W TC275TP_64F200W	393216	16384 to 393216
TC277TC_64F200S TC275TC_64F200W	147456	16384 to 147456
TC267D_40F200S TC265D_40F200W TC264D_40F200W TC260D_40F200 TC264DA_40F200W	98304	16384 to 98304
TC265DC_40F200W TC264DC_40F200W	65536	16384 to 65536
TC237LP_32F200S TC234LP_32F200F TC233LP_32F200F TC237L_32F200S TC234L_32F200F TC233L_32F200F TC234LA_32F200F TC234LX_32F200F TC237LA_32F200S TC237LX_32F200S	131072	16384 to 131072
TC234LP_16F200F TC233L_16F200F TC233LP_16F200F TC233LC_24F133F	131072	16384 to 131072

TC224L_16F133F TC223L_16F133F TC222L_16F133F TC224S_16F133F TC223S_16F133F TC222S_16F133F TC223L_12F133F TC222L_12F133F	98304	16384 to 98304
TC222S_12F133F TC223S_12F133F	65536	16384 to 65536
TC214L_8F133F TC213L_8F133F TC212L_8F133F TC214S_8F133F TC213S_8F133F TC214L_8F133F TC213L_8F133F TC212L_8F133F	65536	16384 to 65536
TC212S_8F133F	49152	16384 to 49152

4.3.14 FlsPBFixedAddress

Table 20 Fixed Address usage for FLS configuration

Syntax:	FlsPBFixedAddress		
Source:	This parameter is microcontroller specific		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value: STD_ON : FLS Configuration to be used is fixed i.e Fls_17_Pmu_ConfigRoot[0] STD_OFF: The user has the option to choose the FLS configuration to be used and pass the same as a parameter to Fls_17_Pmu_Init Default: STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter indicates whether the Fls configuration to be used is decided at link time or changeable by user at runtime.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.15 FlsDebugSupport

Table 21 Debug support for FLS configuration

Syntax:	FlsDebugSupport		
Source:	This parameter is microcontroller specific		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value: STD_ON : Debug support enabled STD_OFF: Debug support disabled Default: STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter indicates whether the Debug support is enabled or disabled for FLS		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4 Container: FlsConfigSet

4.4.1 FlsWaitStateRead

Table 22 FlsWaitStateRead Configuration

Syntax:	FlsWaitStateRead		
Source:	This parameter is driver specific.		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Value: FLS_WAIT_STATE_READACCESS1 FLS_WAIT_STATE_READACCESS2 FLS_WAIT_STATE_READACCESS3 FLS_WAIT_STATE_READACCESS63 Default: FLS_WAIT_STATE_READACCESS9 (for TC27x, TC26x, TC29x, TC23x) FLS_WAIT_STATE_READACCESS4 (for TC22x, TC21x)		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB

Description:	Defines Wait State Configuration for read access. FlsWaitStateRead is derived from the equation $\text{Ceil}(\text{tDF} * \text{fFSI}) - 1$
SFRs Modified	FLASH0_FCON
Scope:	Driver
Dependency:	None

4.4.2 FlsWaitStateErrorCorrection

Table 23 FlsWaitStateErrorCorrection Configuration

Syntax:	FlsWaitStateErrorCorrection		
Source:	This parameter is driver specific.		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value: FLS_WAIT_STATE_ERRORCORRECTION0 FLS_WAIT_STATE_ERRORCORRECTION1 FLS_WAIT_STATE_ERRORCORRECTION2 FLS_WAIT_STATE_ERRORCORRECTION3 FLS_WAIT_STATE_ERRORCORRECTION4 FLS_WAIT_STATE_ERRORCORRECTION5 FLS_WAIT_STATE_ERRORCORRECTION6 FLS_WAIT_STATE_ERRORCORRECTION7 Default : FLS_WAIT_STATE_ERRORCORRECTION1 (for TC27x, TC26x, TC29x, TC23x) FLS_WAIT_STATE_ERRORCORRECTION0 (for TC22x, TC21x)		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Defines Wait State Configuration for error correction. FlsWaitStateErrorCorrection is derived from the equation $\text{Ceil}(\text{tDFECC} * \text{fFSI}) - 1$		
SFRs Modified	FLASH0_FCON		
Scope:	Driver		
Dependency:	None		

4.4.3 FlsCallCycle

Table 24 FlsCallCycle Configuration

Syntax:	FlsCallCycle
Source:	This parameter is defined by AUTOSAR, refer [2] .
File:	Fls_17_Pmu_PBCfg.c
Range:	Value: 100us to 1sec Default: 100ms

Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Cycle time of calls of the flash driver's main function.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.4 FlsDefaultMode

Table 25 FlsDefaultMode Configuration

Syntax:	FlsDefaultMode		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Value: MEMIF_MODE_FAST MEMIF_MODE_SLOW Default: MEMIF_MODE_FAST		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Indicates if the FLS device is in Fast or Slow mode.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.5 FlsAcErase

Table 26 FlsAcErase Configuration

Syntax:	FlsAcErase		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Values: RAM address / Flash address Default: 0x 701011C8		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	<u>FlsAcLoadOnJobStart = STD_ON</u> This parameter specifies the RAM address from which the Erase command cycles will be copied and executed.		

	<i>Note: The user should reserve this address space in the linker (.lsl) file. Refer Chap.2.1.3</i> <u>FlsAcLoadOnJobStart = STD_OFF</u> The value entered here is ignored and the location of Erase command cycles in PFLASH is used
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.4.6 FlsAcWrite

Table 27 FlsAcWrite Configuration

Syntax:	FlsAcWrite		
Source:	This parameter is defined by AUTOSAR, refer [2].		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Values: RAM address / Flash address Default: 0x 70101100		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	<u>FlsAcLoadOnJobStart = STD_ON</u> This parameter specifies the RAM address to which the Write command cycles will be copied and executed. <i>Note: The user should reserve this address space in the linker (.lsl) file. Refer Chap.2.1.3</i> <u>FlsAcLoadOnJobStart = STD_OFF</u> The value entered here is ignored and the location of Write command cycles in PFLASH is used		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.7 FlsJobEndNotification

Table 28 FlsJobEndNotification API Configuration

Syntax:	FlsJobEndNotification
Source:	This parameter is defined by AUTOSAR, refer [2].
File:	Fls_17_Pmu_PBCfg.c
Range:	Values: Fls Job End notification pointer of type FUNCTION_NAME(Selectable) / Address (Loadable) If IFX FEE is used, it should be configured as Fee_JobEndNotification Default: NULL_PTR

Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	User defined Notification Function Pointer of type void <fn_name> (void). This notification function will be called by the FLS Driver on successful completion of the job		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.8 FlsJobErrorNotification

Table 29 FlsJobErrorNotification API Configuration

Syntax:	FlsJobErrorNotification		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Values: Fls Job Error notification of type FUNCTION_NAME (Selectable) / Address (Loadable) If IFX FEE is used, it should be configured as Fee_JobErrorNotification Default: NULL_PTR		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	User defined Notification Function Pointer of type void <fn_name> (void). This notification function will be called by the FLS Driver on cancellation of the job or a failure in executing the job		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.9 FlsMaxReadFastMode

Table 30 FlsMaxReadFastMode Configuration

Syntax:	FlsMaxReadFastMode		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Values: 1 Byte - Sector size Default: 64 Bytes		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	The maximum number of bytes to read in one cycle of the flash driver's		

	job processing in fast mode. Note: User should configure FlsMaxReadFastMode value as greater than FlsMaxReadNormalMode
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.4.10 FlsMaxReadNormalMode

Table 31 FlsMaxReadNormalMode Configuration

Syntax:	FlsMaxReadNormalMode		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Values: 1 Byte - Sector size Default: 32 Bytes		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	The maximum number of bytes to read in one cycle of the flash driver's job processing in normal mode. Note: User should configure FlsMaxReadFastMode value as greater than FlsMaxReadNormalMode		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.11 FlsMaxWriteFastMode

Table 32 FlsMaxWriteFastMode Configuration

Syntax:	FlsMaxWriteFastMode		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	-- (not implemented)		
Range:	Value: Fixed to a page size of 32Bytes. This parameter is not supported and hence DISABLED		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	The maximum number of bytes to write in one cycle of the flash driver's job processing. Note that in the AURIX the write is page wise. So each write operation programs a page (8/32 bytes) based on the size of the data and page start address		
SFRs Modified	None		
Scope:	Driver		

Dependency:	None
--------------------	------

4.4.12 FlsMaxWriteNormalMode

Table 33 FlsMaxWriteNormalMode Configuration

Syntax:	FlsMaxWriteNormalMode		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Value: Fixed to a page size of 32Bytes. This parameter is FIXED and hence not configurable		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	The maximum number of bytes to write in one cycle of the flash driver's job processing. Note that in the AURIX the write is page wise. So each write operation programs a page (8/32 bytes) based on the size of the data and page start address		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.13 FlsProtection

Table 34 FlsProtection Configuration

Syntax:	FlsProtection		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	-		
Range:	Value : 0 to 4294967295 Default : 0 This parameter is unused and hence DISABLED		
Configuration class	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	This parameter is NOT SUPPORTED as protection is handled by FlsLoader.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.14 Container: FlsDemEventParameterRefs

This container contains references to DemEventParameter elements. The DemEventParameter is a container in the DEM module.

4.4.14.1 FLS_E_COMPARE_FAILED

Table 35 FLS_E_COMPARE_FAILED

Syntax:	FLS_E_COMPARE_FAILED		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Reference to DemEventParameter		
Configuration class	Pre-compile	--	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Reference to the DemEventParameter which shall be issued when the error "Flash compare failed (HW)" has occurred		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.14.2 FLS_E_ERASE_FAILED

Table 36 FLS_E_ERASE_FAILED

Syntax:	FLS_E_ERASE_FAILED		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Reference to DemEventParameter		
Configuration class	Pre-compile	--	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Reference to the DemEventParameter which shall be issued when the error "Flash erase failed (HW)" has occurred		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.14.3 FLS_E_READ_FAILED

Table 37 FLS_E_READ_FAILED

Syntax:	FLS_E_READ_FAILED		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Reference to DemEventParameter		
Configuration class	Pre-compile	--	Variant PC
	Link Time	--	--

	Post build	X	Variant PB
Description:	Reference to the DemEventParameter which shall be issued when the error "Flash read failed (HW)" has occurred		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.14.4 FLS_E_WRITE_FAILED

Table 38 FLS_E_WRITE_FAILED

Syntax:	FLS_E_WRITE_FAILED		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Reference to DemEventParameter		
Configuration class	Pre-compile	--	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Reference to the DemEventParameter which shall be issued when the error "Flash write failed (HW)" has occurred		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.14.5 FLS_E_UNEXPECTED_FLASH_ID

Table 39 FLS_E_UNEXPECTED_FLASH_ID

Syntax:	FLS_E_UNEXPECTED_FLASH_ID		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	-		
Range:	Reference to DemEventParameter		
Configuration class	Pre-compile	--	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Reference to the DemEventParameter which shall be issued when the error "Expected Flash ID not Matched" has occurred This parameter is not supported as it concerns only to the external flash driver which is not supported as well.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.14.6 FLS_E_SUSPND_RESUME_TIMEOUT

Syntax:	FLS_E_SUSPND_RESUME_TIMEOUT		
Source:	This parameter is driver specific.		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Reference to DemEventParameter		
Configuration class	Pre-compile	--	Variant PC
	Link Time	--	--
	Post build	X	Variant PB
Description:	Reference to the DemEventParameter which shall be issued when the error "Suspend / Resume Timeout (HW)" has occurred.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.15 Container: FlsExternalDriver

4.4.15.1 FlsSpiReference

Table 40 FlsSpiReference Configuration

Syntax:	FlsSpiReference		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	NA		
Range:	Reference to SPI sequence		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is required only for external Flash driver. Hence not supported.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.16 Container: FlsSectorList

This container includes the container FlsSector

4.4.17 Container: FlsSector

This container contains parameters that describe the sector in terms of size, address etc.

4.4.17.1 FlsSectorStartAddress

Table 41 Start Address of the Fls Sector

Syntax:	FlsSectorStartAddress		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Pre-configured to 0x00. This parameter is FIXED and hence not configurable		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Start address of the Fls Sector		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.17.2 FlsSectorSize

Table 42 Size of the Fls Sector

Syntax:	FlsSectorSize		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Refer Table 43 This parameter is pre-configured and hence NOT editable.		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	Size of the Fls Sector. The value of this parameter is fixed to half of the total size of DF_EEPROM. The value of this parameter is not used to generate FLS_SECTOR_SIZE. Value of FLS_SECTOR_SIZE will be FlsTotalSize / 2.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

Table 43 Range of Fls Sector Size for various devices

Devices	Range
---------	-------

TC299TP_128F300S TC299T_128F300S TC298TP_128F300L TC297TP_128F300S TC297T_128F300S TC290TP_128F300 TC299TX_128F300S TC299TY_128F300S TC297TX_128F300S TC297TY_128F300S TC297T_96F300S TC297TP_96F300S TC297TA_128F300S TC297TB_128F300S TC297TA_64F300S	393216
TC290TC_96F300 TC297TC_96F300S TC299TC_96F300S	98304
TC277TP_64F200S TC277T_64F200S TC275T_64F200W TC275TP_64F200W	196608
TC277TC_64F200S TC275TC_64F200W	73728
TC267D_40F200S TC265D_40F200W TC264D_40F200W TC260D_40F200 TC264DA_40F200W	49152
TC265DC_40F200W TC264DC_40F200W	32768
TC237LP_32F200S TC234LP_32F200F TC233LP_32F200F TC237L_32F200S TC234L_32F200F TC233L_32F200F TC234LA_32F200F TC234LX_32F200F TC237LA_32F200S	65536

TC237LX_32F200S	
TC234LP_16F200F TC233L_16F200F TC233LP_16F200F TC233LC_24F133F	65536
TC224L_16F133F TC223L_16F133F TC222L_16F133F TC224S_16F133F TC223S_16F133F TC222S_16F133F TC223L_12F133F TC222L_12F133F	49152
TC222S_12F133F TC223S_12F133F	32768
TC214L_8F133F TC213L_8F133F TC212L_8F133F TC214S_8F133F TC213S_8F133F TC214L_8F133F TC213L_8F133F TC212L_8F133F	32768
TC212S_8F133F	24576

4.4.17.3 FlsPageSize

Table 44 Size of the Fls Page

Syntax:	FlsPageSize		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	Fls_17_Pmu_Cfg.h		
Range:	Pre-configured to 8. This parameter is FIXED and hence not configurable		
Configuration class	Pre-compile	X	All variants

	Link Time	--	--
	Post build	--	--
Description:	Size of the FIs Page		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.17.4 FIsNumberOfSectors

Table 45 Number of sectors

Syntax:	FIsNumberOfSectors		
Source:	This parameter is defined by AUTOSAR, refer [2] .		
File:	FIs_17_Pmu_Cfg.h		
Range:	Pre-configured to 2. This parameter is FIXED and hence not configurable		
Configuration class	Pre-compile	X	All variants
	Link Time	--	--
	Post build	--	--
Description:	The number of sectors having the above mentioned configuration		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.5 Container: FIsIfxSpecificConfig

This container lists all the IFX specific pre-compile configuration parameters

4.5.1 FIsStateVarStruct

Table 46 FIsStateVarStruct Configuration

Syntax:	FIsStateVarStruct		
Source:	This parameter is driver specific.		
File:	FIs_17_Pmu_Cfg.h		
Range:	Value : User defined structure name		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is used to provide the name of the structure containing the entire global variables specific to the Flash driver.		
SFRs Modified	None		

Scope:	Driver
Dependency:	None

4.5.2 FlsUseEraseSuspend

Table 47 FlsUseEraseSuspend Configuration

Syntax:	FlsUseEraseSuspend		
Source:	This parameter is driver specific.		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value : STD_ON – Erase suspend and resume operations enabled STD_OFF - Erase suspend and resume operations disabled		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is used to enable or disable the erase suspend and resume functionality.		
SFRs Modified	FLASH0_MARD		
Scope:	Driver		
Dependency:	None		

4.5.3 FlsEraseSuspendTimeout

Table 48 FlsEraseSuspendTimeout Configuration

Syntax:	FlsEraseSuspendTimeout		
Source:	This parameter is driver specific.		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value : 0 – 65535 Default Value : 6000		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Timeout parameter for Erase Suspend feature (number of loop counts) Example Calculation: tSPNDD is Max latency taken for Data Flash suspend fFSI is execution speed for data flash operations. $tSPNDD = 12000 / (fFSI \text{ [MHz]})$ micro seconds. Calculation based on TC275 variant with controller clock of 200 MHz. in this exmpale calculation, tSPNDD = 60 micro second. fFSI = 200MHZ		

	Number of instructions in one timeout loop of Driver code = 3. Time taken for execution of 1 instruction = 0.005 microseconds Time taken for execution of 3 instructions = 0.015 microseconds. Loop count to get 60 microseconds = 60 / 0.015 micro seconds = 4000 Value of FlsEraseSuspendTimeout required for succesfull Suspend operation is 4000.
SFRs Modified	None
Scope:	Driver
Dependency:	This parameter is used only when Erase suspend feature is enabled.

4.5.4 FlsIllegalStateNotification

Table 49 FlsIllegalStateNotification Configuration

Syntax:	FlsIllegalStateNotification		
Source:	This parameter is driver specific.		
File:	Fls_17_Pmu_PBCfg.c		
Range:	Values: FLS illegal state notification function pointer of type FUNCTION_NAME(Selectable) / Address (Loadable) When IFX FEE driver is used, this parameter shall be configured as Fee_17_IllegalStateNotification. Default: NULL_PTR		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter is a pointer to a notification function, which is called when FLS reaches an Illegal state. Illegal state is reached whenever OPER flag is set. Illegal state here means that FLS is not able to proceed. No more FLS request shall be triggered; Reset of the system is recommended.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	When IFX FEE driver is used, this parameter shall be configured as Fee_17_IllegalStateNotification.		

4.6 Container: FlsPublishedInformation

This container lists all the published parameters.

4.6.1 FlsAcSizeErase

Table 50 Erase Access Code size

Syntax:	FLS_17_PMU_AC_SIZE_ERASE
Type:	-

File:	Fls_17_Pmu.xdm
Value:	200
Description:	Number of bytes in RAM needed for the erase flash access code

4.6.2 FlsAcSizeWrite

Table 51 Write Access Code size

Syntax:	FLS_17_PMU_AC_SIZE_WRITE
Type:	-
File:	Fls_17_Pmu.xdm
Value:	300
Description:	Number of bytes in RAM needed for the Write flash access code

4.6.3 FlsEraseTime

Table 52 Erase Time

Syntax:	FLS_17_PMU_ERASE_TIME
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	1000000 (us)
Description:	Maximum time to erase one complete flash sector.

4.6.4 FlsWriteTime

Table 53 Write time

Syntax:	FLS_17_PMU_WRITE_TIME
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	150 (us)
Description:	Maximum time to write one complete flash page.

4.6.5 FlsErasedValue

Table 54 Erased Value

Syntax:	FLS_17_PMU_ERASED_VALUE
Type:	-
File:	Fls_17.xdm
Value:	0
Description:	The contents of an erased flash memory cell.

4.6.6 FlsSpecifiedEraseCycles

Table 55 Erase Cycles Supported

Syntax:	FLS_17_PMU_SPECIFIED_ERASE_CYCLES
Type:	-
File:	Fls_17.xdm
Value:	125000
Description:	Number of erase cycles specified for the flash device

4.6.7 FIsAcLocationErase

Table 56 FLS Erase Command cycles location

Syntax:	FLS_17_PMU_AC_LOCATION_ERASE
Type:	-
File:	Not implemented (The location is specified using FIsAcErase)
Value:	0
Description:	Location of the erase command cycles. This parameter is not used. Instead use the parameter FIsAcErase

4.6.8 FIsAcLocationWrite

Table 57 FLS Write Command cycles location

Syntax:	FLS_17_PMU_AC_LOCATION_WRITE
Type:	-
File:	Not implemented (The location is specified using FIsAcWrite)
Value:	0
Description:	Location of the write command cycles. This parameter is not used. Instead use the parameter FIsAcWrite

4.6.9 FIsExpectedHwId

Table 58 Hardware ID

Syntax:	FLS_17_PMU_EXPECTED_HW_ID
Type:	-
File:	Not implemented (Relevant external flash drivers)
Value:	0
Description:	Unique identifier of the hardware device that is expected by this driver (the device for which this driver has been implemented). This parameter is NOT USED as it is relevant for external flash drivers

4.7 Container: CommonPublishedInformation

This container lists all the common published parameters.

4.7.1 ArMajorVersion

Table 59 AUTOSAR Major Version

Syntax:	FLS_17_PMU_AR_RELEASE_MAJOR_VERSION
----------------	-------------------------------------

Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	4
Description:	Major version of supported AUTOSAR release

4.7.2 ArMinorVersion

Table 60 AUTOSAR Minor Version

Syntax:	FLS_17_PMU_AR_RELEASE_MINOR_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	0
Description:	Minor version of supported AUTOSAR release

4.7.3 ArPatchVersion

Table 61 AUTOSAR Patch Version

Syntax:	FLS_17_PMU_AR_RELEASE_REVISION_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	3
Description:	Patch (revision) version of supported AUTOSAR release

4.7.4 SwMajorVersion

Table 62 Software Major Version

Syntax:	FLS_17_PMU_SW_MAJOR_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	Major version of the driver
Description:	Major version of the driver

4.7.5 SwMinorVersion

Table 63 Software Minor Version

Syntax:	FLS_17_PMU_SW_MINOR_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	Minor version of the driver
Description:	Minor version of the driver

4.7.6 SwPatchVersion

Table 64 Software Patch Version

Syntax:	FLS_17_PMU_SW_PATCH_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	Patch version of the driver
Description:	Patch version of the driver

4.7.7 ModuleId

Table 65 Module Id

Syntax:	FLS_17_PMU_MODULE_ID
Type:	uint16
File:	Fls_17_Pmu_Cfg.h
Value:	92
Description:	Module Identification number of FLS driver

4.7.8 VendorId

Table 66 Vendor Id

Syntax:	FLS_17_PMU_VENDOR_ID
Type:	uint16
File:	Fls_17_Pmu_Cfg.h
Value:	17
Description:	Identification number of the Vendor (Infineon Technologies)

4.7.9 VendorApilInfix

Table 67 Vendor API Infix

Syntax:	Fls_17_Pmu
Type:	-
File:	Fls_17_Pmu.xdm
Value:	Fls_17_Pmu
Description:	Vendor API Infix of the FLS driver

4.7.10 Release

Table 68 Release

Syntax:	-
Type:	-
File:	Fls_17_Pmu.xdm
Value:	Aurix derivative used for the implementation
Description:	Aurix derivative used for the implementation

4.8 Derived Configuration Parameters

These parameters represent the additional information required by the Driver and are automatically calculated by the tool.

4.8.1 FLS_IFX_FEE_USED

Table 69 FLS_IFX_FEE_USED Configuration

Syntax:	FLS_IFX_FEE_USED		
Source:	This parameter is a derived configuration parameter		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value : STD_ON/ STD_OFF STD_ON: IFX FEE Driver is used within the same configuration workspace STD_OFF: IFX FEE Driver is NOT used in the same configuration workspace Default : STD_OFF		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter indicates if an IFX FEE Driver is being used.		
Scope:	Driver		
Dependency:	None		

4.8.2 FLS_SECTOR_SIZE

Table 70 FLS_SECTOR_SIZE Configuration

Syntax:	FLS_SECTOR_SIZE		
Source:	This parameter is a derived configuration parameter		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value : FlsTotalSize / 2		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter indicates the size of one logical sector used by the double sector algorithm of FEE. Note: FLS_SECTOR_SIZE is not generated from FlsSectorSize.		
Scope:	Driver		
Dependency:	None		

4.8.3 FLS_17_PMU_CONFIG_COUNT

Table 71 FLS_17_PMU_CONFIG_COUNT Configuration

Syntax:	FLS_17_PMU_CONFIG_COUNT		
Source:	This parameter is a derived configuration parameter		
File:	Fls_17_Pmu_Cfg.h		
Range:	Value : Number of configuration sets under FlsConfigSet		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter indicates the number of configuration sets present under the container FlsConfigSet		
Scope:	Driver		
Dependency:	None		

5 Published parameters

This section describes the parameters published by the FLS Driver.

5.1 VendorId

Table 72 VendorId

Syntax:	FLS_17_PMU_VENDOR_ID
Type:	Text
File:	Fls_17_Pmu_Cfg.h, File Header Comments
Value:	17
Description:	Vendor Id of the supplier

5.2 ModuleId

Table 73 ModuleId

Syntax:	FLS_17_PMU_MODULE_ID
Type:	Macro, type cast to uint16
File:	Fls_17_Pmu_Cfg.h
Value:	92 (0x5C)
Description:	This macro gives the Flash driver module ID as described by AUTOSAR : Wp1.1.2 Basic Software Module List

5.3 SwMajorVersion

Table 74 SwMajorVersion

Syntax:	FLS_17_PMU_SW_MAJOR_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h

Value:	Refer Fls_17_Pmu_Cfg.h
Description:	Major version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The MAJOR_VERSION is incremented if the Driver is not downwards compatible any more (e.g. existing API changed)

5.4 SwMinorVersion

Table 75 SwMinorVersion

Syntax:	FLS_17_PMU_SW_MINOR_VERSION
Type:	Macro - #define
File:	Fls_17_Pmu_Cfg.h
Value:	Refer Fls_17_Pmu_Cfg.h
Description:	Minor version number of the vendor specific implementation of the Driver. The numbering is vendor specific. MINOR_VERSION is incremented if the Driver is still downwards compatible (e.g. new functionality added)

5.5 SwPatchVersion

Table 76 SwPatchVersion

Syntax:	FLS_17_PMU_SW_PATCH_VERSION
Type:	Macro - #define
File:	Fls_17_Pmu_Cfg.h
Value:	Refer Fls_17_Pmu_Cfg.h
Description:	Patch level version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The PATCH_VERSION is incremented if the Driver is still upwards and downwards compatible (e.g. bug fixed)

5.6 AR_RELEASE_MAJOR_VERSION

Table 77 FLS Driver AUTOSAR Requirement Major Version

Syntax:	FLS_17_PMU_AR_RELEASE_MAJOR_VERSION
Type:	Macro – #define
File:	Fls_17_Pmu_Cfg.h
Value:	4
Description:	Major version number of AUTOSAR specification on which the driver implementation is based on.

5.7 AR_RELEASE_MINOR_VERSION

Table 78 FLS Driver AUTOSAR Requirement Minor Version

Syntax:	FLS_17_PMU_AR_RELEASE_MINOR_VERSION
Type:	Macro - #define
File:	Fls_17_Pmu_Cfg.h

Value:	0
Description:	Minor version number of AUTOSAR specification on which the driver implementation is based on.

5.8 AR_RELEASE_REVISION_VERSION

Table 79 FLS Driver AUTOSAR Requirement Patch Version

Syntax:	FLS_17_PMU_AR_RELEASE_REVISION_VERSION
Type:	Macro - #define
File:	Fls_17_Pmu_Cfg.h
Value:	3
Description:	Patch version number of AUTOSAR specification on which the driver implementation is based on.

6 API Documentation

6.1 API Type Definitions

This section describes all the type definitions that are exported by Flash driver. These type definitions are either used for configuration or for API parameter passing.

6.1.1 Type definition Fls_ConfigType

Table 80 Type definition Fls_ConfigType

Syntax:	Fls_ConfigType
Type:	Struct
File:	Fls_17_Pmu.h
Range:	This configuration structure contains the initialization data structure of the Flash Module.
Description:	This type definition is used to configure overall Flash configuration. A pointer to such a structure is provided to the module's initialization routine for configuration of the Flash EEPROM Emulation module.

6.1.2 Type definition MemIf_StatusType

Table 81 Type definition MemIf_Status Type

Syntax:	MemIf_StatusType
Type:	Enum
File:	MemIf_Types.h
Range:	MEMIF_UNINIT: The flash driver is not initialized or not usable. This shall be the default value after reset. This status shall have the value 0. MEMIF_IDLE: The flash driver is ready to accept a write or erase command.

	MEMIF_BUSY: The flash driver is performing a job (read, write or erase).
Description:	This is a status value returned by the API service Fls_17_Pmu_GetStatus(). This type is imported from MemIf_Types.h

6.1.3 Type definition MemIf_JobResultType

Table 82 Type definition MemIf_JobResultType

Syntax:	MemIf_JobResultType
Type:	Enum
File:	MemIf_Types.h
Range:	MEMIF_JOB_OK: The last write or erase job has been finished successfully. This shall be the default value after reset. This status shall have the value 0. MEMIF_JOB_PENDING: The flash driver is performing a write or erase job. The meaning of this status is equal to MEMIF_BUSY. MEMIF_JOB_CANCELED: The last job has been aborted by the Fls_17_Pmu_Cancel() function. MEMIF_JOB_FAILED: The last job has failed. MEMIF_BLOCK_INCONSISTENT: The last block read has inconsistent data. MEMIF_BLOCK_INVALID: The last block read has invalid contents.
Description:	This is the job result returned by the API service Fls_17_Pmu_GetJobResult(). This type is imported from MemIf_Types.h

6.1.4 Type definition MemIf_DataType

Table 83 Type definition MemIf_DataType

Syntax:	MemIf_DataType
Type:	Uint8
File:	MemIf_Types.h
Range:	0..255, 1 Byte
Description:	Type of application data buffer elements. For consistency checks, this data type shall be declared as a type definition.

6.1.5 Type definition Fls_LengthType

Table 84 Type definition Fls_LengthType

Syntax:	Fls_LengthType
Type:	Uint32
File:	Fls_17_Pmu.h
Range:	32bit
Description:	Specifies the number of bytes (type: MemIf_DataType) to read, write or erase.

6.1.6 Type definition Fls_AddressType

Table 85 Type definition Fls_AddressType

Syntax:	Fls_AddressType
Type:	Uint32
File:	Fls_17_Pmu.h
Range:	32 bit address
Description:	This value is used as address offset for accessing flash memory. The flash device has 0 as first address. A flash base address will be added by the driver for obtaining the physical address

6.2 API Functions

This chapter describes the API interfaces provided by FLS Driver.

6.2.1 Fls_17_Pmu_Init

Table 86 Specification of API Fls_17_Pmu_Init

Service name:	Fls_17_Pmu_Init	
Syntax:	void Fls_17_Pmu_Init(const Fls_ConfigType* ConfigPtr)	
Service ID:	0	
Sync/ Async:	Sync	
Reentrancy:	Non Re-entrant	
Parameters (in):	ConfigPtr	Pointer to the Configuration Set.
Parameters (out):	None	
Return value:	None	
Description:	Service for Flash initialization. This function initializes the flash driver (software) and all flash memory relevant registers (hardware) with parameters provided in the given configuration set. After having finished the module initialization the flash driver state is set to MEMIF_IDLE and the flash job result is set to MEMIF_JOB_OK.	
Caveats:	None	
Configuration:	None	
DET:	When FlsPBFixedAddress is STD_OFF The DET FLS_E_PARAM_CONFIG is raised if the input parameter ConfigPtr is NULL. When FlsPBFixedAddress is STD_ON The DET FLS_E_PARAM_CONFIG is raised if the input parameter ConfigPtr is NOT equal to address of Fls_17_Pmu_ConfigRoot[0]. i.e. Fls driver is configured to use the configuration in Fls_17_Pmu_ConfigRoot[0] only	
DEM:	None	
Implementation Comments:	Initialization of SFR and Global Variables: The Init function initializes all SFR and global variables that are needed by the Module. However, if the initial value of the SFR is the reset value, the init functions does not reinitialize them.	

	This is done in order to reduce initialization time and avoid double initialization. The module expects the init function to be called after microcontroller reset. Reads the OPER error status and saves it for use in other APIs wherever necessary. If OPER error had occurred during FLASH startup, then FLS illegal state notification function will be called (if configured in FLSIllegalStateNotification) and reaches the illegal state. When FlsPBFixedAddress is STD_ON The Driver sets an internal variable to TRUE in order to indicate that the Fls_17_Pmu_Init has been called. Note: DFLASH uncorrectable ECC error Trap is disabled in this API. It should not be activated. If it has to be activated then customer trap handler should be written such that it returns back to FLS driver. FEE (user of FLS) handles the ECC error condition.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModelInitApiEnable is 'False'.

6.2.2 Fls_17_Pmu_Erase

Table 87 Specification of API Fls_17_Pmu_Erase

Service name:	Fls_17_Pmu_Erase	
Syntax:	Std_ReturnType Fls_17_Pmu_Erase(Fls_AddressType TargetAddress, Fls_LengthType Length)	
Service ID:	1	
Sync/ Async:	Async	
Reentrancy:	Non Re-entrant	
Parameters (in):	TargetAddress	Target (logical) address in flash memory. This address offset will be added to the flash memory base address.
	Length	Number of bytes to erase
Parameters (out):	None	
Return value:	E_OK	Erase Command was accepted.
	E_NOT_OK	Erase Command was not accepted.
Description:	This service will update the driver internal variable with JobType, initiate an erase job, set the driver status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK. The erase job is executed asynchronously within the flash driver's main function. A flash memory block starting from FLS_BASE_ADDRESS + TargetAddress of size Length will be erased.	

	The Erase job will be started by the Erase function if the hardware is idle and OPER error had not occurred	
Caveats:	None	
Configuration:	-	
DET:	FLS_E_PARAM_ADDRESS	The Address does not align to the Data Flash (Both Banks) sector Address.
	FLS_E_PARAM_LENGTH	Address + Length not aligned to either of DFLASH Sector 0/1 End Address
	FLS_E_UNINIT	Flash Driver is not initialized.
	FLS_E_BUSY	Flash Driver is busy.
DEM:	FLS_E_ERASE_FAILED	The erase operation is failed due to HW error
Implementation Comments:	This API checks the current status of OPER error as well as the OPER error status which was saved before; If OPER error is detected in any one of them, then FLS illegal state notification function will be called (if configured in FlsIllegalStateNotification) and the API is exited without starting the erase operation i.e., E_NOT_OK will be returned and FLS reaches the illegal state. If OPER error is not detected, then this API starts the erase job and additional DEM errors are detected in this function. Since the implementation of this FLS driver assumes that the FEE driver uses double sector algorithm for EEPROM emulation, this API allows only the following Erase options: • Erase only first half of DF_EEPROM • Erase only second half of DF_EEPROM • Erase complete DF_EEPROM Hence the parameters passed to this API should correspond to one of the above cases only.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'False'.	

6.2.3 Fls_17_Pmu_Write

Table 88 Specification of API Fls_17_Pmu_Write

Service name:	Fls_17_Pmu_Write
Syntax:	Std_ReturnType Fls_17_Pmu_Write(Fls_AddressType TargetAddress, const uint8 *SourceAddressPtr, Fls_LengthType Length)
Service ID:	2
Sync/ Async:	Async
Reentrancy:	Non Re-entrant

Parameters (in):	SourceAddressPtr	Pointer to source data buffer
	TargetAddress	Target address in flash memory. This address offset will be added to the flash memory base address.
	Length	Number of bytes to write.
Parameters (out):	None	
Return value:	E_OK	Write Command was accepted.
	E_NOT_OK	Write Command was not accepted.
Description:	Service for writing one or more complete flash pages. This service will copy the given parameters to driver internal variables, initiate a write job, set the driver status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK. The write job shall be executed asynchronously within the flash driver's main function. A flash memory block starting from FLS_BASE_ADDRESS + TargetAddress of size Length will be programmed with the data provided via SourceAddressPtr. The write job is initiated in this function. i.e., the programming of the first page is already started in this function, if the hardware is idle and OPER error had not occurred.	
Caveats:	<i>Note: SourceAddressPtr should point to address which is even aligned. For best optimization, it should point to 64bit aligned address i.e. the buffer should be 64bit aligned</i>	
Configuration:	None	
DET:	FLS_E_PARAM_ADDRESS	The Address does not align to the Data Flash page Address.
	FLS_E_PARAM_LENGTH	Address + Length is beyond either of DFLASH Sector 0/1 End Address
	FLS_E_PARAM_DATA	The value of Pointer to source data buffer is NULL.
	FLS_E_UNINIT	Flash Module is not initialized.
	FLS_E_BUSY	Flash Module is busy.
	FLS_E_VERIFY_ERASE_FAILED	The requested page is not erased before Write operation.
DEM:	FLS_E_WRITE_FAILED	The erase operation is failed due to HW error
Implementation Comments:	This API checks the current status of OPER error as well as the OPER error status which was saved before; If OPER error is detected in any one of them, then FLS illegal state notification function will be called (if configured in FLSIllegalStateNotification) and the API is exited without starting the write operation i.e., E_NOT_OK will be returned and FLS reaches the illegal state. If OPER error is not detected, then this API starts the write job and additional DEM errors are detected in this function. The Target Address should align to the data flash page start address. The allowed values in linear space are 0 and multiples of 8/32 (Page Size = 8/32bytes) The possible values for length should be multiples of 8/32 bytes. An implicit check of the page (as required for FLS_E_VERIFY_ERASE_FAILED) is not done incase FLS_IFX_FEE_USED = ON (i.e FEE Driver from IFX is used)	

I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'False'.
------------------	--

6.2.4 Fls_17_Pmu_Cancel

Table 89 Specification of API Fls_17_Pmu_Cancel

Service name:	Fls_17_Pmu_Cancel	
Syntax:	void Fls_17_Pmu_Cancel(void)	
Service ID:	3	
Sync/ Async:	Sync	
Reentrancy:	Non Re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	Service for canceling an ongoing flash write or erase job. This function aborts the pending jobs so that directly after returning from this function a new job can be accepted by the Driver. The function will reset the driver's internal job processing variables (like address, length and data pointer) and set the driver state to FLS_IDLE. The routine will set the job result to MEMIF_JOB_CANCELED, if the job result currently has the value MEMIF_JOB_PENDING. Otherwise it will leave the job result unchanged.	
Caveats:	None	
Configuration:	FlsCancelApi = ON	
DET:	FLS_E_UNINIT	Flash module is not initialized.
DEM:	None	
Implementation Comments:	In case an erase job or Write job is ongoing the function does not wait till the completion of the ongoing job. The Error Notification is called if a job is cancelled.	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.5 Fls_17_Pmu_MainFunction

Table 90 Specification of API Fls_17_Pmu_MainFunction

Service name:	Fls_17_Pmu_MainFunction
Syntax:	void Fls_17_Pmu_MainFunction(void)
Service ID:	6
Sync/ Async:	Sync

Reentrancy:	Non Re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This function performs the processing of the flash read, write and erase jobs. When a job has been initiated, this function has to be called cyclically until the job is finished. The function might also be called cyclically if no job is pending. In this case the function will return without any action. The flash driver main function will only process as much data in one call cycle as statically configured for the current job type (read, write or erase). If interrupt controlled job processing is supported and enabled, the interrupt service routine located in Fls_Irq.c will reset the interrupt flag, check for errors reported by the underlying hardware, reload the hardware finite state machine for the next round of the pending job or call the appropriate notification routine if the job is finished or aborted. The function Fls_17_Pmu_MainFunction is still required for processing of jobs without hardware interrupt support (e.g. read jobs). If development error detection is enabled, the routine will check if the driver has been initialized. If not an error will be reported to the DET with the error code FLS_E_UNINIT. In case of ongoing erase/write jobs, this API checks the status of OPER error; If OPER error is detected, then FLS illegal state notification function will be called (if configured in FlsIllegalStateNotification) and the ongoing erase/write job will be terminated; Job error notification function will be called if configured and FLS reaches the illegal state. <i>Note: The clearing of the DFLASH status is done when the operation is finished successfully or has failed.</i> <i>Note: DET should be DISABLED while running FLS/FEE in bootloader to avoid immediate timeout.</i>	
Caveats:	None	
Configuration:	FLS_USE_INTERRUPTS	
DET:	FLS_E_UNINIT	Flash Module is not initialized.
	FLS_E_VERIFY_ERASE_FAILED	The requested page is not erased before Write operation.
	FLS_E_VERIFY_WRITE_FAILED	The requested page is not programmed with the value pointed by Source pointer after write operation
	FLS_E_TIMEOUT	The requested job (Erase/Write) could not be finished within the required time.
DEM:	FLS_E_ERASE_FAILED	Erase Job Failed due to HW Error.
	FLS_E_WRITE_FAILED	Write Job Failed due to HW Error.
	FLS_E_READ_FAILED	Read Job Failed due to HW Error.

		This case happens if a double bit error is detected.
	FLS_E_COMPARE_FAILED	Compare Job Failed due to HW Error. This case happens if a double bit error is detected.
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'False'.	

6.2.6 Fls_17_Pmu_Read

Table 91 Specification of API Fls_17_Pmu_Read

Service name:	Fls_17_Pmu_Read	
Syntax:	Std_ReturnType Fls_17_Pmu_Read(Fls_AddressType SourceAddress, MemIf_DataType *TargetAddressPtr, Fls_LengthType Length)	
Service ID:	7	
Sync/ Async:	Async	
Reentrancy:	Non Re-entrant	
Parameters (in):	SourceAddress	Source address in flash memory. This address offset will be added to the flash memory base address.
	TargetAddressPtr	Pointer to target data buffer
	Length	Number of bytes to read.
Parameters (out):	None	
Return value:	E_OK	Read Command was accepted.
	E_NOT_OK	Read Command was not accepted.
Description:	Service for reading from flash memory. This service will copy the given parameters to driver internal variables, initiate a read job, set the driver status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK. The read job will be executed asynchronously within the flash driver's main function. A continuous flash memory block starting from FLS_BASE_ADDRESS + SourceAddress of size Length will be copied to the buffer pointed to by TargetAddressPtr.	
Caveats:	None	
Configuration:	None	
DET:	FLS_E_PARAM_ADDRESS	The Address is not a valid DFLASH Address.
	FLS_E_PARAM_LENGTH	Address not aligned to either of DFLASH Sector 0/1 End Address

	FLS_E_PARAM_DATA	The value of Pointer to source data buffer is NULL.
	FLS_E_UNINIT	Flash Module is not initialized.
	FLS_E_BUSY	Flash Module is busy.
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.7 Fls_17_Pmu_Compare

Table 92 Specification of API Fls_17_Pmu_Compare

Service name:	Fls_Compare	
Syntax:	Std_ReturnType Fls_17_Pmu_Compare (Fls_AddressType SourceAddress, uint8 *TargetAddressPtr, Fls_LengthType Length)	
Service ID:	8	
Sync/ Async:	Async	
Reentrancy:	Non Re-entrant	
Parameters (in):	SourceAddressPtr	Target address in flash memory. This address offset will be added to the flash memory base address.
	TargetAddress	Pointer to source data buffer
	Length	Number of bytes to compare
Parameters (out):	None	
Return value:	E_OK	Compare Command was accepted.
	E_NOT_OK	Compare Command was not accepted.
Description:	Service for comparing the contents of an area of flash memory with that of an application data buffer. This service will copy the given parameters to driver internal variables, initiate a compare job, set the driver status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK. The compare job will be executed asynchronously within the flash driver's main function. A continuous flash memory block starting from FLS_BASE_ADDRESS + SourceAddress of size Length will be compared to the buffer pointed to by TargetAddressPtr	
Caveats:	None	
Configuration:	FlsCompareApi = ON	
DET:	FLS_E_PARAM_ADDRESS	The Address is not a valid DFLASH Address.
	FLS_E_PARAM_LENGTH	Address + Length not aligned to a valid Flash page end address.
	FLS_E_PARAM_DATA	The value of Pointer to source data buffer is NULL.
	FLS_E_UNINIT	Flash Module is not initialized.
DEM:	None	
Implementation Comments:	This API can be included upon defining FLS_COMPARE_API as ON.	

I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.
------------------	--

6.2.8 Fls_17_Pmu_SetMode

Table 93 Specification of API Fls_17_Pmu_SetMode

Service name:	Fls_17_Pmu_SetMode	
Syntax:	void Fls_17_Pmu_SetMode (MemIf_ModeType Mode)	
Service ID:	9	
Sync/ Async:	Sync	
Reentrancy:	Non Re-entrant	
Parameters (in):	MEMIF_MODE_SLOW	Slow read access
	MEMIF_MODE_FAST	Fast read access
Parameters (out):	None	
Return value:	None	
Description:	Sets the read mode to either fast or slow based on the parameter Mode	
Caveats:	None	
Configuration:	FlsSetModeApi = ON	
DET:	None	
DEM:	None	
Implementation Comments:	This API can be included upon defining FLS_SET_MODE_API as ON. FAST or SLOW mode impacts only Read/Compare operations. Based on the currently active mode, the number of bytes to be read from Flash in each call of Fls_17_Pmu_MainFunction is taken from the configuration.	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.9 Fls_17_Pmu_GetVersionInfo

Table 94 Specification of API Fls_17_Pmu_GetVersionInfo

Service name:	Fls_17_Pmu_GetVersionInfo	
Syntax:	void Fls_17_Pmu_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)	
Service ID:	16	
Sync/ Async:	Sync	
Reentrancy:	reentrant	
Parameters (in):	VersionInfoPtr	Pointer to where to store the version information of this module
Parameters (out):	None	
Return value:	None	
Description:	This service returns the version information of this module. The version information includes: Module Id Vendor Id Instance Id	

	Vendor specific software version numbers.	
Caveats:	None	
Configuration:	This function shall be pre compile time configurable On/Off by the configuration parameter: FlsVersionInfoApi	
DET:	FLS_E_PARAM_POINTER	Parameter VersionInfoPtr is NULL.
DEM:	None	
Implementation Comments:	The Version Info is not updated if the FLS_E_PARAM_POINTER DET is raised. This API is implemented as a macro	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.10 Fls_17_Pmu_ReadWordsSync

Table 95 Specification of API Fls_17_Pmu_ReadWordsSync

Service name:	Fls_17_Pmu_ReadWordsSync	
Syntax:	<pre>Std_ReturnType Fls_17_Pmu_ReadWordsSync (Fls_AddressType SourceAddress, const uint32 *TargetAddressPtr, uint32 Length)</pre>	
Service ID:	17	
Sync/ Async:	Sync	
Reentrancy:	Non Re-entrant	
Parameters (in):	SourceAddress	Source address in flash memory. This address offset will be added to the flash memory base address.
	TargetAddressPtr	Pointer to target data buffer
	Length	Number of words to be read.
Parameters (out):	None	
Return value:	E_OK	Read Command was accepted.
	E_NOT_OK	Read Command was not accepted.
Description:	This is an IFX specific API and not listed in the SWS. It is a service for reading from flash memory synchronously. This service will copy the given parameters to driver internal variables, initiate a read job, set the driver status to MEMIF_BUSY and read the required words of data from the DFlash only if no operation is ongoing in the DFlash Bank. This service also clears the Flash status register incase of errors. Note: Reading data that overlaps from sector0 to sector1 is not supported as this API is used only by FEE and FEE makes sure that it always reads data within a sector.	
Caveats:	The source and target addresses should be - valid - word aligned	

Configuration:	None	
DET:	None	
DEM:	FLS_E_READ_FAILED	Read Job Failed due to HW Error. This case happens if a double bit error is detected.
Implementation Comments:	Available only if FLS_IFX_FEE_USED = STD_ON	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.11 Fls_17_Pmu_GetStatus

Table 96 Specification of API Fls_17_Pmu_GetStatus

Service name:	Fls_17_Pmu_GetStatus
Syntax:	MemIf_StatusType Fls_17_Pmu_GetStatus(void)
Service ID:	4
Sync/ Async:	Sync
Reentrancy:	Re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	MemIf_StatusType
Description:	This service will return the driver state as MEMIF_BUSY : If either or both banks are busy MEMIF_IDLE : If both banks are idle MEMIF_UNINIT : If FLS Driver is not initialized
Caveats:	None
Configuration:	FlsGetStatusApi = ON
DET:	None
DEM:	None
Implementation Comments:	None
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.

6.2.12 Fls_17_Pmu_GetJobResult

Table 97 Specification of API Fls_17_Pmu_GetJobResult

Service name:	Fls_17_Pmu_GetJobResult
Syntax:	MemIf_JobResultType Fls_17_Pmu_GetJobResult(void)
Service ID:	5
Sync/ Async:	Sync
Reentrancy:	Re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	MemIf_JobResultType
Description:	This service will return the result of the last job synchronously. The erase, write and read services share the same job result, i.e. only the result of the last job can be queried. Every new job that has been

	accepted by the flash driver overwrites the job result with MEMIF_JOB_PENDING.
Caveats:	None
Configuration:	FlsGetJobResultApi = ON
DET:	None
DEM:	None
Implementation Comments:	The function should be called after call to Fls_17_Pmu_Init. The status returned before call to Fls_17_Pmu_Init is MEMIF_JOB_OK
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.

6.2.13 Fls_17_Pmu_CompareWordsSync

Table 98 Specification of API Fls_17_Pmu_CompareWordsSync

Service name:	Fls_17_Pmu_CompareWordsSync	
Syntax:	Std_ReturnType Fls_17_Pmu_CompareWordsSync (Fls_AddressType SourceAddress, const uint32 *TargetAddressPtr, uint32 Length)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	SourceAddress	Source address in flash memory. This address offset will be added to the flash memory base address
	TargetAddressPtr	Pointer to target data buffer
	Length	Number of words to be read
Parameters (out):	None	-
Return value:	E_OK	Compare command accepted and comparison is successful.
	E_NOT_OK	Compare unsuccessful.
Description:	This is an IFX specific API and not listed in the SWS. It is a service for comparing from flash memory synchronously. Note: Comparing data that overlaps from sector0 to sector1 is not supported as this API is used only by FEE and FEE makes sure that it always compares data within a sector.	
Caveats:	The source and target addresses should be 1. valid 2. word aligned	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	Available only if FLS_IFX_FEE_USED = STD_ON	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.14 Fls_17_Pmu_VerifyErase

Table 99 Specification of API Fls_17_Pmu_VerifyErase

Service name:	Fls_17_Pmu_VerifyErase	
Syntax:	Std_ReturnType Fls_17_Pmu_VerifyErase (uint32 BankNum, uint32 * const UnerasedWordlineAddressPtr, uint8 * const UnerasedWordlineCountPtr)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	BankNum	DFlash Bank Num on which Synchronous erase verify needs to be performed
Parameters (out):	UnerasedWordlineAddressPtr	Pointer to first unerased word-line address
	UnerasedWordlineCountPtr	Pointer to unerased word-lines count
Return value:	E_OK E_NOT_OK	Erase verification Command was accepted and passed Erase verification Command was not accepted or failed with more than one Unerasable WL -
Description:	This is an IFX specific API and not listed in the SWS. It is a service for verifying erase of a sector synchronously.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	Available only if FLS_IFX_FEE_USED = STD_ON	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.15 Fls_17_Pmu_VerifySectorErase

Table 100 Specification of API Fls_17_Pmu_VerifySectorErase

Service name:	Fls_17_Pmu_VerifySectorErase	
Syntax:	Std_ReturnType Fls_17_Pmu_VerifySectorErase (uint32 BankNum, uint32 * const UnerasedWordlineAddressPtr, uint8 * const UnerasedWordlineCountPtr uint8 Sector)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	

Parameters (in):	BankNum	DFlash Bank Num on which Synchronous erase verify needs to be performed
Parameters (out):	UnerasdWordlineAddressPtr	Pointer to first unerased word-line address
	UnerasdWordlineCountPtr	Pointer to unerased word-lines count
	Sector	Logical Sector number to be verified
Return value:	E_OK E_NOT_OK	Erase verification Command was accepted and passed Erase verification Command was not accepted or failed with more than one Unerasable WL -
Description:	This is an IFX specific API and not listed in the SWS. It is a service for verifying erase of a logical sub sector synchronously	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	Available only if FLS_IFX_FEE_USED = STD_ON	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.16 Fls_17_Pmu_SuspendErase

Table 101 Specification of API Fls_17_Pmu_SuspendErase

Service name:	Fls_17_Pmu_SuspendErase	
Syntax:	Std_ReturnType Fls_17_Pmu_SuspendErase (uint32 BankNum)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	BankNum	Sector on which erase is to be suspended
Parameters (out):	-	-
Return value:	E_OK	Erase Suspend successful or Erase is already suspended.
	E_NOT_OK	Erase Suspend failed or this API is called when Erase is not on-going
Description:	This is an IFX specific API and not listed in the SWS. It is a service for suspending an ongoing erase of a sector.	
Caveats:	None	
Configuration:	FlsUseEraseSuspend	
DET:	None	
DEM:	FLS_E_SUSPND_RESUME_TIMEOUT	
Implementation Comments:	Available only if FLS_USE_ERASESUSPEND = STD_ON	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided	

configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModeRuntimeApiEnable is 'True'.
This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'True'.
This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'False'.

6.2.17 Fls_17_Pmu_ResumeErase

Table 102 Specification of API Fls_17_Pmu_ResumeErase

Service name:	Fls_17_Pmu_ResumeErase	
Syntax:	Std_ReturnType Fls_17_Pmu_ResumeErase (uint32 BankNum)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	BankNum	Sector on which erase is to be resumed
Parameters (out):	-	-
Return value:	E_OK	Erase resume Command was accepted and passed or Erase was not suspended when this API was called
	E_NOT_OK	Erase resume Command was not accepted or failed
Description:	This is an IFX specific API and not listed in the SWS. It is a service for resuming a suspended erase of a sector.	
Caveats:	None	
Configuration:	FlsUseEraseSuspend	
DET:	None	
DEM:	FLS_E_SUSPND_RESUME_TIMEOUT	
Implementation Comments:	Available only if FLS_USE_ERASESUSPEND = STD_ON. This API checks the current status of OPER error as well as the OPER error status which was saved before; If OPER error is detected in any one of them, then FLS illegal state notification function will be called (if configured in FlsIllegalStateNotification) and the API is exited without executing the resume erase operation i.e., E_NOT_OK will be returned and FLS reaches the illegal state. If OPER error is not detected, then this API executes the resume erase operation and reports DEM when a Resume timeout is occurred. If OPER error occurs upon execution of resume erase, then FLS illegal state notification function will be called (if configured in FlsIllegalStateNotification) and FLS reaches the illegal state.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'False'.	

6.2.18 Fls_17_Pmu_GetBankStatus

Table 103 Specification of API Fls_17_Pmu_GetBankStatus

Service name:	Fls_17_Pmu_GetBankStatus	
Syntax:	MemIf_StatusType Fls_17_Pmu_GetBankStatus (uint32 BankNum)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Reentrant	
Parameters (in):	BankNum	- DFlash sector number whose status needs to be determined
Parameters (out):	-	-
Return value:	MEMIF_BUSY MEMIF_IDLE MEMIF_UNINIT	The sector is busy The sector is idle Fls Driver is not initialized
Description:	This is an IFX specific API and not listed in the SWS. Returns the state of a sector.	
Caveats:	None	
Configuration:	FlsGetStatusApi = ON	
DET:	None	
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.19 Fls_17_Pmu_CancelNonEraseJobs

Table 104 Specification of API Fls_17_Pmu_CancelNonEraseJobs

Service name:	Fls_17_Pmu_CancelNonEraseJobs	
Syntax:	void Fls_17_Pmu_CancelNonEraseJobs (void)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	-	-
Parameters (out):	-	-
Return value:	-	-

Description:	This is an IFX specific API and not listed in the SWS. Service for canceling ongoing flash jobs except the erase job. This function aborts the pending jobs (except erase job) so that directly after returning from this function a new job can be accepted by the Driver. The function shall reset the driver's internal job processing variables (like address, length and data pointer) and set the driver state to FLS_IDLE. The routine will set the job result of the ongoing job types (except erase job) to MEMIF_JOB_CANCELED, if the job result currently has the value MEMIF_JOB_PENDING. Otherwise it will leave unchanged the job result.
Caveats:	None
Configuration:	None
DET:	None
DEM:	None
Implementation Comments:	None
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.

6.2.20 Fls_17_Pmu_GetNotifCaller

Table 105 Specification of API Fls_17_Pmu_GetNotifCaller

Service name:	Fls_17_Pmu_GetNotifCaller	
Syntax:	Fls_17_Pmu_Job_Type Fls_17_Pmu_GetNotifCaller(void)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	-	-
Parameters (out):	-	-
Return value:	FLS_NO_JOB FLS_ERASE_JOB FLS_WRITE_JOB FLS_READ_JOB FLS_COMPARE_JOB FLS_CANCEL_JOB	No notification was called Notification for Erase job Notification for Write job Notification for Read job Notification for Compare job Notification for Cancel job
Description:	This is an IFX specific API and not listed in the SWS. Returns the FLS Job that raised the notification	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	Available only if FLS_IFX_FEE_USED = STD_ON	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.2.21 Fls_17_Pmu_GetOperStatus

Table 106 Specification of API Fls_17_Pmu_GetOperStatus

Service name:	Fls_17_Pmu_GetOperStatus	
Syntax:	Std_ReturnType Fls_17_Pmu_GetOperStatus(void)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	-	-
Parameters (out):	-	-
Return value:	E_OK	No OPER error
	E_NOT_OK	OPER error had occurred
Description:	This is an IFX specific API and not listed in the SWS. Returns whether OPER error had occurred or not	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	Upper layers can call this function to know whether OPER error has occurred or not. Upon reaching the illegal state (illegal state notification function will be called if configured in FlsIllegalStateNotification parameter), the upper layer can call this function to know whether illegal state is reached due to OPER error or for some other reason. If OPER error had occurred, no more FLS/FEE requests shall be called. Reset of the system is recommended.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter FlsRunningInUser0Mode is 'True' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter FlsRunningInUser0Mode is 'False' and FlsUserModeRuntimeApiEnable is 'False'.	

6.2.22 Fls_17_Pmu_ControlTimeoutDet

Table 107 Specification of API Fls_17_Pmu_ControlTimeoutDet

Service name:	Fls_17_Pmu_ControlTimeoutDet	
Syntax:	void Fls_17_Pmu_ControlTimeoutDet(uint8 Param)	
Service ID:	NA	
Sync/Async:	Sync	
Reentrancy:	Non-Reentrant	
Parameters (in):	Param	= 0 for disable; = 1 for re-enable.
Parameters (out):	-	-
Return value:	-	-

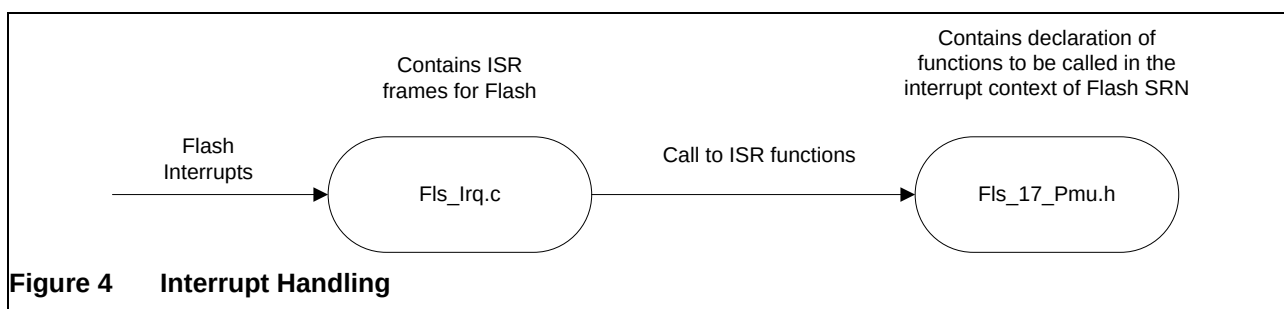
Description:	This is an IFX specific API and not listed in the SWS. This is the function to disable and re-enable detection / reporting of timeout DET
Caveats:	Fls_17_Pmu_Init() should be called before. This API shall be called only when FLS driver status is IDLE. This API shall not be called when an erase job is in suspended status.
Configuration:	None
DET:	None
DEM:	None
Implementation Comments:	This API will be defined as an empty function when development error detection is disabled. By default timeout detection and reporting will be enabled after Fls_17_Pmu_Init.
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.

6.3 Debug Macros

There are debug code written within the driver to simulate certain HW conditions, however these code fragments are encapsulated with a DEBUG macro i.e. the final binary will not have these instructions. The macros are like IFX_FLS_DEBUG, IFX_FLS_DEBUG01, IFX_FLS_DEBUG02 and so on.

6.4 Interrupt Handling

The interrupt handling within the Flash Module is depicted as under.



The Interrupt Node “SRC_PMU00” is used and has to be configured.

The functions to be called in the interrupt context are explained in the following sections.

6.4.1 Fls_17_Pmu_Isr

Table 108 **Specification of API Fls_17_Pmu_Isr**

Service name:	Fls_17_Pmu_Isr
Syntax:	void Fls_17_Pmu_Isr() (void)
Service ID:	None
Sync/ Async:	Sync
Reentrancy:	non reentrant
Parameters (out):	None
Return value:	None

Description:	This interrupt is mapped to the node: SRC_PMU00. This function performs the Write and Erase Job Handling.	
Caveats:	None	
Configuration:	FlsUseInterrupts = STD_ON	
DET:	FLS_E_VERIFY_ERASE_FAILED	The requested page is not erased before Write operation/ Erase operation did not erase the flash correctly
	FLS_E_VERIFY_WRITE_FAILED	The requested page is not programmed with the value pointed by Source pointer after write operation
DEM:	FLS_E_ERASE_FAILED	Erase Job Failed due to HW Error.
	FLS_E_WRITE_FAILED	Write Job Failed due to HW Error.
Implementation Comments:	None	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.5 Exclusive Area (EA) Information

The exclusive areas (critical sections) are provided for any of the following reason:

- 1) To protect shared resources from the module interrupts. The resources are used in ISR's.
- 2) To protect shared resources from another API / same API.

6.5.1 Init

Table 109 Specification of EA Init

API Names:	Fls_17_Pmu_Init
Resource Name:	FLASH0_FCON register
ISR Name:	NA
Comment:	This EA is entered and exited using the APIs SchM_Enter_Fls_17_Pmu_Init and SchM_Exit_Fls_17_Pmu_Init This register is ENDINIT protected and hence the protection unlocking and subsequent locking should be completed before the Watchdog Timeout, hence the critical section is created

6.5.2 Write

Table 110 Specification of EA Write

API Names:	Fls_17_Pmu_Write
Resource Name:	Write Command Cycles
ISR Name:	NA
Comment:	This EA is entered and exited using the APIs SchM_Enter_Fls_17_Pmu_Write and SchM_Exit_Fls_17_Pmu_Write To provide protection for write command cycles from same API / another API (Fls_17_Pmu_Erase)

6.5.3 Erase

Table 111 Specification of EA Erase

API Names:	Fls_17_Pmu_Erase
Resource Name:	Erase Command Cycles
ISR Name:	NA
Comment:	This EA is entered and exited using the APIs SchM_Enter_Fls_17_Pmu_Erase and SchM_Exit_Fls_17_Pmu_Erase To provide protection for erase command cycles from same API / another API (Fls_Write)

6.5.4 Main

Table 112 Specification of EA Main

API Names:	Fls_IMainErase, Fls_IMainWrite
Resource Name:	Erase/Write Command Cycles
ISR Name:	Fls_17_Pmu_Isr
Comment:	This EA is entered and exited using the APIs SchM_Enter_Fls_17_Pmu_Main and SchM_Exit_Fls_17_Pmu_Main To provide protection for write/erase command cycles against concurrent access. i.e. protect shared resources from the module interrupts

6.5.5 Resume Erase

Table 113 Specification of EA Resume Erase

API Names:	Fls_17_Pmu_ResumeErase
Resource Name:	Resume Erase Command Cycles
ISR Name:	NA
Comment:	This EA is entered and exited using the APIs SchM_Enter_Fls_17_Pmu_ResumeErase and SchM_Exit_Fls_17_Pmu_ResumeErase To provide protection for ResumeErase command cycles against concurrent access. i.e. protect shared resources from the module interrupts

7 Data Description

The below table lists all the global and static variables used (elements of the data structure):

Table 114 Global Data Description

Parameter	Type/Range	Description
FlsReadAddress	uint32	Address to be read
FlsWriteAddress	uint32	Address to be written
FlsEraseTimeoutCycleCount	uint32	Handles the main function call count during Erase job
FlsWriteTimeoutCycleCount	uint32	Handles the main function call count during

		Write job
FlsReadLength	uint32	Size of data to be read
FlsWriteLength	uint32	Size of data to be written
FlsReadBufferPtr	uint8*	Pointer to read buffer
FlsWriteBufferPtr	uint8*	Pointer to write buffer
FlsJobResult	MemIf_JobResultType	Flash job result
FlsMode	MemIf_ModeType	Flash mode
NotifCaller	uint8	Job which raised the notification
JobStarted	Bit field – uint8	Job state
FlsJobType	uint8[2]	Job type
DriverInitialised	uint8	Driver initialization state
FlsPver	uint8	Program verification error status
FlsOper	uint8	Operation error status
FlsTimeoutErr	uint8	Job which encountered timeout error
FlsTimeoutControl	uint8	Timeout handling status

8 Error Classification

The Driver provides pre-compile switches so that production errors are reported to DEM module only if their reporting is enabled.

Table 115 Error Classification

Error Code	Error Description	Relevance (Development/ Production)	Value (hex)
FLS_E_PARAM_CONFIG	API Service called with wrong parameter. Config is a null pointer	Development	0x1
FLS_E_PARAM_ADDRESS	API service called with wrong/inconsistent Address(s)	Development	0x2
FLS_E_PARAM_LENGTH	API service called with wrong/inconsistent length(s)	Development	0x3
FLS_E_PARAM_DATA	API service called with wrong/inconsistent Data(s)	Development	0x4
FLS_E_UNINIT	API service called without module initialization	Development	0x5
FLS_E_BUSY	API service called while Driver is still busy	Development	0x6
FLS_E_VERIFY_ERASE_FAILED	Erase Verification (blank check) failed	Development	0x7
FLS_E_VERIFY_WRITE_FAILED	Write Verification (compare) failed	Development	0x8
FLS_E_TIMEOUT	Timeout occurred for Write/Erase job	Development	0x9
FLS_E_PARAM_POINTER	Pointer parameter is NULL	Development	0xA
FLS_E_ERASE_FAILED	Error (SQER,DFOPER,PROER) occurred during erase operation	Production	Assigned by DEM
FLS_E_WRITE_FAILED	Error (SQER,DFOPER,PROER) occurred during write operation failed	Production	Assigned by DEM
FLS_E_READ_FAILED	Non-correctable ECC error occurred when reading from DFLASH	Production	Assigned by DEM

FLS_E_COMPARE_FAILED	Non-correctable ECC error occurred when reading from DFLASH or the data in DFLASH not matching with the buffer contents	Production	Assigned by DEM
FLS_E_SUSPEND_RESUME_TIMEOUT	Timeout Error During Erase Suspend/ Resume operation	Production	Assigned by DEM

9 Example Usage

9.1 Configuration of the Driver

The configuration of the driver involves the following important steps.

1. Configuration of System Clock "fSYS". This Configuration is done using the MCU module
2. Configuration of FLS Driver. Refer [Section 4](#)
3. If FlsAcLoadOnJobStart is configured as true, then reserve 300 bytes of RAM for loading Write command sequence and 200 bytes of RAM for loading Erase command sequence. Refer [Section 2.1.3](#).

9.2 Using the Flash Services

9.2.1 Initializing the Driver

The [Code Listing 2](#) shows the steps involved in the initialization of the Flash Driver.

Code Listing 2 Driver Initialization

```

030:  /* Configure FLS Module Interrupt Priority */
031:  #if FLS_USE_INTERRUPTS == ON
032:  IrqPmu_Init();
033:  #endif
034:
035:  /* Initialization of flash module */
036:  Fls_17_Pmu_Init(&Fls_17_Pmu_ConfigRoot[0]);

```

9.2.2 Flash Operations

The [Code Listing 3](#) shows an example of the steps involved in erasing, writing and reading a Data Flash Bank (Bank0) after initialization of Flash.

Code Listing 3 Flash Operations

```

037:  #define FLS_BANK0_LINEAR (0U)
038:  #define FLS_BANK_SIZE    (0x30000U)
039:  #define FLS_PAGESIZE    (8U)
040:
041:  uint8 Test_ProgData[2 * FLS_PAGESIZE];
042:  uint8 Test_ReadData[2 * FLS_PAGESIZE];
043:
044:  /* Erase DFLASH BANK 0 */
045:  ReturnVal = Fls_17_Pmu_Erase( FLS_BANK0_LINEAR, FLS_BANK_SIZE);
046:  /* Wait for completion of Erase Job*/
047:  while(Fls_17_Pmu_GetJobResult() == MEMIF_JOB_PENDING)
048:  {}

```

```

049:
050:  /* Write first 2 pages of DFLASH BANK 0. Initialize Test_ProgData with
    dummy values */
051:  ReturnVal = Fls_17_Pmu_Write(Test_ProgData,
    FLS_BANK0_LINEAR, (2*FLS_PAGESIZE));
052:  /* Wait for completion of Write Job*/
053:  while(Fls_17_Pmu_GetJobResult() == MEMIF_JOB_PENDING)
054:  {}
055:  /* Read the first two pages */
056:  ReturnVal = Fls_17_Pmu_Read(FLS_BANK0_LINEAR ,Test_ReadData, (2 *
    FLS_PAGESIZE));
057:  /* Wait for completion of Read Job*/
058:  while(Fls_17_Pmu_GetJobResult() == MEMIF_JOB_PENDING)
059:  {}
060:

```

10 Assumptions and Limitations

10.1 Assumptions and Deviations from the Software Specification

S.No.	Assumptions and Deviations
1.	Protection settings is not used in the FLS Driver as it is best handled by the FlsLoader Driver and hence the parameter FlsProtection is NOT supported
2.	FlsMaxWriteFastMode is NOT supported as write is performed for 8/32 bytes depending on the data size and page start address

10.2 Limitations

1. FlsMaxWriteNormalMode and FlsMaxWriteFastMode are fixed to 32 bytes.
2. Incase of Fls_17_Pmu_Write the SourceAddressPtr containing the content to be written should be even aligned.
4. Since FlsLoader also accesses PMU0, the access of PMU0 by FlsLoader and access of DF0 by FEE/FLS driver should not be concurrent.
5. Although the Fls_17_Pmu_Cancel API is Synchronous, hardware may be still busy after returning from Fls_17_Pmu_Cancel API due to already issued Flash Erase or Write command sequence. In such scenarios any new job requested will be accepted but will be executed with dealy (new job will be started by the subsequent main function called after hardware becomes free).

.Hardware Features Not Supported

1. The Program Flash is not supported as it is assumed that the driver is only for EEPROM Flash Emulation.
2. DFSBER, DFDBER, DFTBER handling is not supported as it violates FEE016, MEMIF010 of AUTOSAR R4.0 Rev3

www.infineon.com